Sharded Blockchains

Matthieu Pigaglio

Thesis submitted in partial fulfillment of the requirements for

the Degree of Doctor in Applied Sciences

October 2026

ICTEAM

Louvain School of Engineering

Université catholique de Louvain

Louvain-la-Neuve

Belgium

Thesis Committee:

Pr. Wonderful **Supervisor** (Advisor)	UCLouvain/ICTEAM, Belgium
Pr. Good **Doctor**	UCLouvain/ICTEAM, Belgium
Pr. Another **Guy**	Another University, World

Contents

Table of Contents **i**

I Background and State of the Art **1**

1 Blockchain Fundamentals **3**

1.1 Distributed Ledgers 4

1.2 Consensus 12

1.2.1 BFT Variants 14

1.2.2 Proof-of-Work 16

1.2.3 Proof-of-Stake 18

1.3 Ethereum Proof-of-Stake 20

1.3.1 Validators 22

1.3.2 Slots and Epochs 24

1.3.3 Committees and Attestations 26

1.3.4 Fork-Choice Rule 28

1.3.5 Rewards and Safeguards 31

1.4 Transaction and State Models 34

1.4.1 UTXO Model 35

1.4.2 Account-Based Model 37

1.5 Smart Contracts and Execution Environments 40

1.5.1 The Ethereum Virtual Machine 42

1.5.2 Alternative Execution Environments 46

2 The Scalability Problem **49**

2.1 Throughput, Latency, and Full-Node Cost Bottlenecks 51

2.2 Exploiting Workload Properties 57

2.2.1 Parallel Transaction Execution 59

2.2.2 Light Nodes and Stateless Clients 62

2.3 On-Chain Scaling: Sharding 65

2.3.1 Network Sharding 68

2.3.2 Transaction Sharding 70

2.3.3 State Sharding 72

2.3.4 Existing Sharded Blockchains and Their Limitations . 76

2.4 Off-Chain Scaling: Layer 2 85

2.4.1 Rollups 86

2.4.1.1 Optimistic Rollups 88

2.4.1.2 Zero-Knowledge Rollups 89

2.4.2 State Channels 90

2.4.3 Sidechains 91

2.4.4 Existing Layer 2 and Data Availability Solutions and Their Limitations 93

3 Synthesis and Motivation **103**

3.1 Open Challenges 104

3.2 Common Requirements for L1 and L2 Sharding 108

3.3 Positioning of the Contributions 110

Part I

Background and State of the Art

Blockchain Fundamentals | 1

This chapter covers the blockchain concepts that the rest of the thesis depends on. It is not a survey: each section is here because a later part needs it, and approaches to scalability are left to Chapter 2. Generic material is kept short. Ethereum gets more space, because all three contributions target it: Part II analyzes its transaction workload, Part III designs state sharding for it, and Part IV fits data availability sampling into its consensus.

The chapter goes from the general model to Ethereum's specifics. Section 1.1 describes a distributed ledger as a replicated state machine. It introduces blocks, hash chaining, Merkle trees, the roles that nodes play, and the full replication that limits throughput. Section 1.2 explains how nodes agree on the order of transactions, covering the fault and network models, BFT pro-

tocols, proof-of-work and proof-of-stake. Section 1.3 then details Ethereum's proof-of-stake: validators, slots and epochs, committees and attestations, the fork-choice rule, and the rewards and penalties that keep validators honest. Part III builds on these committees and penalties, and Part IV has to fit within the timing of a slot. Section 1.4 compares the UTXO and account-based models. The state model determines what a transaction accesses, and so when two transactions conflict and how the state can be split between shards. Finally, Section 1.5 presents smart contracts and the Ethereum Virtual Machine. It shows how calls between contracts create the dependencies between accounts that Parts II and III study.

1.1 Distributed Ledgers

A distributed ledger is an append-only log of transactions, replicated by nodes that do not trust each other. Bitcoin [Nak08], launched in 2009, was the first such ledger to work in an open setting. Its transactions transfer a native cur-

rency. Ethereum [Woo14] followed in 2015 and let transactions call programs, smart contracts, whose code and storage live in the replicated state. This section covers the parts of this model that the rest of the thesis uses; how nodes agree on the content of the log is left to Section 1.2.

State machine replication. A ledger is a form of state machine replication [Sch90]. Each node holds a copy of a state S (in the simplest case, the balance of every account), and a transaction tx updates it through a transition function, $S' = \delta(S, tx)$. Nodes that start from the same *genesis* state S_0 and apply the same transactions $tx_1, \ldots, tx_n$ all end up with

$$S_n = \delta(\ldots \delta(\delta(S_0, tx_1), tx_2) \ldots, tx_n).$$

This only works if δ is deterministic. Smart contract execution environments (Section 1.5) are built around this requirement: programs get no local clock,

no floating-point arithmetic and no access to external services, since any of these could give different results on different machines. The state itself never needs to be exchanged, because any node can recompute it from the log. Nodes only have to agree on the order of transactions, and that is the job of consensus.

Blocks and hash chaining. Transactions enter the log in batches called blocks. A block has a body, the ordered list of its transactions, and a header with metadata: typically the hash of the parent header, a timestamp, and short commitments to the content of the block, such as the root of a Merkle tree over its transactions and, in some ledgers, a commitment to the state after the block is executed.

Since each header contains the hash of its parent, blocks form a hash chain back to genesis. The idea predates blockchains and was first used to timestamp digital documents [HS91]. If the hash function is collision resistant, the

most recent header fixes the whole history. Altering a transaction in an old block changes that block's hash, hence the parent hash stored in its child, and so on up to the latest header. Any node that knows a recent header sees the change. Hash chaining, however, only makes a rewrite detectable. Nothing stops an attacker from building a different chain that is internally consistent, and choosing among several valid chains is again a matter for consensus (Section 1.2).

Merkle trees. The commitments in the header are usually roots of Merkle trees [Mer87]. Leaves are hashes of data items, and each inner node hashes its children, so the root depends on every leaf. To prove that an item is in the tree, it is enough to give the sibling hashes on the path from its leaf to the root. The proof grows logarithmically with the number of leaves, and anyone who knows the root can check it. The state can be committed the same way. When the header holds the root of a tree over all state entries

(account balances, contract storage), any single entry can be proven against it. How the state is laid out in practice depends on the transaction and state model of the ledger, the topic of Section 1.4.

Permissioned and permissionless ledgers. Ledgers also differ in who maintains them. In a permissioned ledger such as Hyperledger Fabric [And+18], participants are known and authenticated in advance. Classical Byzantine fault-tolerant protocols (Section 1.2.1) then apply directly, since they assume a fixed set of n nodes of which at most f are faulty. Public blockchains such as Bitcoin are permissionless. Anyone can join or leave, and an identity is just a public key, free to generate. Nothing prevents one participant from creating many identities (a Sybil attack [Dou02]), so votes cannot be counted per identity. Voting power is tied instead to a resource that is hard to obtain: computation in proof-of-work (Section 1.2.2), locked stake in proof-of-stake (Section 1.2.3).

We only consider permissionless ledgers in this thesis. This constrains the mechanisms we add, namely the placement of state on shards in SOAPS (Part III) and the assignment of data to nodes in PANDAS (Part IV). Both must work with open membership and with some participants acting maliciously. Any node must be able to verify their outcome, and participants who would gain from a particular outcome must not be able to bias it.

Node roles. Nodes do not all do the same work [CBC22]. A full node checks each new block by re-executing its transactions and comparing the result with the commitments in the header. It stores the current state, sometimes with a short window of recent ones. Joining nodes rarely replay the chain from genesis. They start from a recent checkpoint, download the matching state, and validate blocks from there; under proof-of-stake, the checkpoint must come from a trusted source, an issue tied to weak subjectivity (Section 1.2.3). An archive node additionally keeps every historical state, so it can answer

queries such as the balance of an account at some past block, at the cost of a much larger disk footprint. Validators (miners under proof-of-work) are full nodes that also propose and vote on blocks; under proof-of-stake, they are the participants that have locked stake (Section 1.2.3). At the other end, light clients follow only block headers and check individual values with Merkle proofs [CBC22]. They execute nothing and rely on most consensus participants being honest (Section 2.2.2).

Peer-to-peer network. Nodes form a peer-to-peer overlay in which each keeps a bounded number of connections. Transactions and blocks spread by gossip: a node that receives a valid message forwards it to some of its neighbors [Dem+87]. Peers find each other through a discovery protocol, often a distributed hash table such as Kademlia [MM02]. To keep bandwidth in check, practical gossip protocols push full messages to a few neighbors and only announce them, with short metadata, to the rest.

The network matters for security. How fast blocks propagate bounds how fast they can be produced before security degrades [GKL15; Ger+16], which is why block intervals are set well above the propagation delay.

Full replication. In the model so far, every full node stores the whole state and re-executes every transaction of every block. No node has to trust another, but adding nodes adds no capacity. Throughput is capped by what one node, run by an ordinary participant, can download, store and execute. Larger or more frequent blocks would lift the cap, at a price. Running a node would become more expensive, so fewer participants could afford it, and blocks would propagate more slowly, which means more forks and weaker security [Ger+16]. Chapter 2 discusses this problem and the main ways around it. This thesis builds on one of them, sharding, where each node handles only a subset of the transactions and of the data.

1.2 Consensus

Nodes that apply the same transactions in the same order reach the same state (Section 1.1). A consensus protocol makes them agree on that order, block after block, even when some nodes are faulty or malicious and the network delays messages [LSP82].

Fault and network models. Blockchain consensus protocols generally assume *Byzantine* faults: a faulty node may do anything. It may stop responding, send wrong messages, collude with other faulty nodes, or *equivocate*, i.e., send conflicting messages to different nodes, such as votes for two different blocks at the same height. Assumptions about the network vary more. The synchronous model guarantees delivery within a known bound Δ. The asynchronous model gives no bound at all, and Fischer, Lynch, and Paterson proved that in this model no deterministic protocol can guarantee agree-

ment if even a single node may crash [FLP85]. Partial synchrony sits in between. The network may be asynchronous for a while but eventually delivers messages within a bound, and agreement is then possible with fewer than a third of Byzantine nodes [DLS88]. Longest-chain protocols assume synchrony [GKL15]; most BFT protocols target partial synchrony [CL99; Yin+19].

Safety and liveness. Two properties are expected from consensus. *Safety* means that honest nodes never confirm two conflicting blocks, so a confirmed transaction is never reverted. *Liveness* means that the protocol keeps making progress, so a valid transaction from an honest client is eventually included. When the network can be partitioned, the two pull against each other. A protocol that keeps producing blocks on both sides of a partition cannot also promise that those blocks never conflict. One answer is to run two layers: a chain that keeps growing under any conditions, and a slower finality mechanism that confirms a prefix of it only once enough participants agree [NTT21].

Ethereum is built this way (Section 1.3).

Sybil resistance. Because votes in a permissionless ledger are weighted by a costly resource and not counted per identity (Section 1.1), security assumptions are stated in terms of that resource: honest participants hold a majority of the computing power in proof-of-work (Section 1.2.2), or a majority of the stake in proof-of-stake, more than two thirds when finality comes from a BFT-style vote as in Ethereum (Section 1.2.3). Classical BFT protocols (Section 1.2.1) offer no Sybil resistance on their own. In practice they are often run on top of proof-of-stake, by a committee chosen according to stake.

1.2.1 BFT Variants

Classical Byzantine fault-tolerant (BFT) protocols, from PBFT [CL99] onward, assume a fixed, known set of n nodes, at most f of them Byzantine, with $n \geq 3f + 1$. Decisions need a *quorum* of $2f + 1$ nodes. Two quorums overlap

in at least $f + 1$ nodes, hence in at least one honest node, and an honest node never votes for two conflicting values. Two conflicting decisions can thus never both reach a quorum. In PBFT, a leader proposes a value and the nodes agree on it in three phases: pre-prepare, prepare, and commit. A faulty or slow leader is replaced through a *view change* [CL99]. Unlike in longest-chain protocols, finality is immediate: a decision is never reverted as long as the bound on f holds.

The cost is communication. PBFT has every node send its votes to every other node, $O(n^2)$ messages per decision, which in practice limits the number of participants. HotStuff [Yin+19] brings this down to $O(n)$: the leader collects the votes and combines them into one threshold signature. Aggregatable signatures such as BLS [BLS01; Bon+03] serve the same purpose, turning thousands of votes into a single signature checked in one operation.

Sharded blockchains such as OmniLedger [Kok+18] and RapidChain [ZMR18]

run a BFT protocol inside each shard, among the committee assigned to it. Committee size is then a trade-off. Small committees agree faster, but they are more likely, by chance, to have more than a third of malicious members. Section 2.3 comes back to this.

1.2.2 Proof-of-Work

Proof-of-work was first proposed against spam, as a small computation required for each email [DN92]. Bitcoin turned it into a way to elect who appends the next block [Nak08]. A miner produces a valid block by finding a nonce for which the hash of the block header falls below a target. No shortcut is known, so miners try values one by one, and the chance of finding the next block is proportional to one's share of the total computing power. The target is adjusted periodically to keep the block rate roughly constant, at one block every ten minutes on average in Bitcoin.

When two miners find a block at nearly the same time, the chain splits

into two branches, or *forks*. A *fork-choice rule* decides which branch to follow. Bitcoin's is the longest chain rule (more precisely, the chain with the most accumulated work), and miners build on the branch it selects. GHOST [SZ15] follows the heaviest subtree instead. Blocks on discarded branches then still count, and the protocol stays secure at higher block rates. Proof-of-stake fork-choice rules later built on this idea (Section 1.3.4).

Finality under these rules is only probabilistic. A heavier branch can always revert a block, but the probability drops exponentially with the number of blocks on top of it, hence the usual advice to wait for several confirmations. The Bitcoin backbone analysis [GKL15] proves the protocol secure when honest miners hold a majority of the computing power and blocks propagate quickly relative to the block interval. Larger or more frequent blocks cause more forks, and the work spent on discarded branches no longer protects the chain [Ger+16]. Throughput cannot simply be raised by tuning these

parameters, and Chapter 2 starts from this observation. Proof-of-work is also energy-hungry, since security grows with the computation miners spend.

1.2.3 Proof-of-Stake

In proof-of-stake, participants called *validators* lock an amount of currency, their *stake*, and their weight in the protocol is proportional to it. They earn rewards for proposing blocks and voting correctly. Ouroboros [Kia+17] and Algorand [Gil+17] both come with formal security proofs, but take different routes. Ouroboros keeps a longest-chain structure and picks block producers according to stake. Algorand draws small committees by cryptographic sortition, and each committee decides on a block with a Byzantine agreement protocol.

Since the protocol holds the stake, it can punish misbehavior directly. A validator that equivocates, for instance by signing two different blocks for the same position, produces two signed messages that together prove the fault

and identify who committed it [She+21a]. Anyone can put this proof in a block, and the protocol then destroys part of the validator's stake. This is called *slashing*. Security rests on economics: an attack needs a large share of the stake and puts that stake at risk. SOAPS (Part III) relies on rewards and slashing, and Section 1.3.5 describes how Ethereum implements them.

At each step, a proof-of-stake protocol must pick the proposer of the next block and the committees that vote on it, using randomness produced by the protocol itself [Gil+17]. If this randomness is predictable long in advance, attackers can prepare against the selected nodes. If participants can influence it, they can steer the selection toward positions that benefit them, and this has been shown to work in practice [Nag+25]. **Matthieu** *CHECK: that Nagy et al. show the manipulation in practice (on mainnet) and not only in principle.* SOAPS and PANDAS inherit the problem, since both assign tasks to nodes from this randomness (Section 1.3.3).

Two further problems are specific to proof-of-stake. Signing a block costs nothing, so a validator could vote on every competing branch; slashing exists to rule out this *nothing-at-stake* behavior. Slashing has a blind spot, though. Once validators have withdrawn, their old keys can still sign an alternative history from far in the past (a *long-range attack*). A node joining the network cannot tell such a history from the real one using the genesis block alone, and needs a recent checkpoint from a source it trusts, an assumption called *weak subjectivity*.

1.3 Ethereum Proof-of-Stake

All three contributions target Ethereum, and two of them depend directly on its consensus. SOAPS (Part III) builds on its committees and penalties, and PANDAS (Part IV) must fit in the timing of its slots. This section goes into more detail than Section 1.2 for that reason. Unless stated otherwise, parameters come from the consensus specifications [Etha].

Ethereum launched with proof-of-work and moved to proof-of-stake in September 2022, in the upgrade known as *the Merge*. Since then, an Ethereum node runs two programs. The *consensus client* runs the *beacon chain*: it manages the validator set, assigns validators to committees, and decides which blocks belong to the chain and which are final [Sch+22]. The *execution client* holds the state and the pool of pending transactions, and executes the transactions of each block in the Ethereum Virtual Machine (Section 1.5.1). The two talk over a local interface, the Engine API, through which the consensus client asks the execution client to build or check the *execution payload*, the list of transactions carried by a beacon block. The beacon chain itself is inherited from earlier sharding designs, where it was meant to coordinate many shard chains (Section 2.3). The system model of SOAPS follows that original design.

1.3.1 Validators

Becoming a validator requires a deposit of at least 32 ETH in a deposit contract and running both a consensus and an execution client. Each validator has an *effective balance*, derived from its actual balance, which sets its voting weight and its chance of being picked as proposer or aggregator. Until the Pectra upgrade in May 2025, the effective balance was capped at 32 ETH, which forced large stakeholders to run many validators. Pectra raised the cap to 2048 ETH for validators that opt into compounding withdrawal credentials, so stake can now be consolidated into fewer validators. Stake remains concentrated all the same: staking pools and liquid staking services control a large share of it, spread over many separate validators.

Validators have three regular duties. When selected as the proposer of a slot, they propose a block. Once per epoch, they *attest*, i.e., they sign a vote on the current head of the chain and on the latest checkpoints (Sec-

tion 1.3.3). Some are also chosen as aggregators and combine the attestations of their committee into one signature. Since the Altair upgrade, there is also a *sync committee* of 512 validators, renewed every 256 epochs (about 27 hours), which signs each block header. Light clients can then follow the chain without tracking the whole validator set (Section 2.2.2).

Validators enter and leave the active set through queues. A *churn limit* bounds how many can be activated or exit per epoch. The validator set cannot change too quickly, and in particular an attacker cannot gain a large weight in a short time. The active set is on the order of a million validators. **Matthieu** *CHECK: current number of active validators at submission (consolidation since Pectra lowers it).* At this scale, a classical BFT protocol in which every validator exchanges votes with every other (Section 1.2.1) is out of the question. Ethereum relies instead on committees and signature aggregation, described below.

1.3.2 Slots and Epochs

Time is divided into *slots* of 12 seconds, and 32 slots make an *epoch* of 6.4 minutes (Figure 1.1). Each slot has a single proposer, which may publish one block. If it does not, the slot stays empty.

Proposers are drawn at random, weighted by effective balance, from a value called the RANDAO mix. Every proposer includes in its block a signature over the current epoch number, and this signature is mixed into the RANDAO value. The mix at the end of an epoch seeds the selection of proposers and committees for a later epoch. A proposer cannot choose its contribution, since its signature is fixed in advance. It can withhold its block, though, and its contribution is then skipped. The last proposers of an epoch can exploit this to pick among several possible seeds and bias future proposer selection in their favor [Nag+25]. The selection is also public before the slots it concerns. Nodes can prepare, but so can an attacker, who knows which node to target

at what time (e.g., with a denial-of-service attack).

Each slot follows a fixed schedule. At 0 s, the proposer publishes its block, which then propagates through the network. The validators in the committee of the slot attest to the head of the chain as soon as they have received and validated the block, and at the latest at 4 s. At 8 s, aggregators publish the aggregated attestations of their committee. A validator thus has at most 4 seconds to receive and check a block before voting. **Matthieu** *CHECK: slot timing at submission (enshrined PBS and shorter-slot proposals may change the 12 s / 4 s schedule).* If the block arrives late or cannot be checked in time, the validator votes for the previous head, and the block will likely be left out of the chain. In PANDAS (Part IV), validators must also verify that the data attached to the block is available, and this check has to fit in the same 4 seconds.

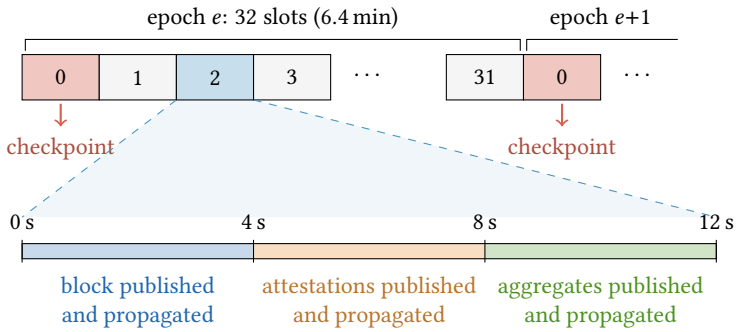


Figure 1.1: Slots and epochs in Ethereum proof-of-stake. An epoch has 32 slots of 12 seconds, and the first slot of each epoch serves as its checkpoint. Within a slot, the block is published at 0 s, attestations at 4 s, and aggregated attestations at 8 s.

1.3.3 Committees and Attestations

At each epoch, the active validators are shuffled with the RANDAO seed and split into committees. Each slot gets between 1 and 64 committees, and each validator sits in exactly one committee per epoch, so it attests exactly once per epoch. When there are enough validators, the shuffle keeps every committee at or above 128 members, above the minimum of 111 recommended for committee safety. With around a million validators, committees are in fact several times larger, with several hundred members each.

The shuffle is deterministic and uses only public data, the seed and the

list of active validators. Any node can recompute the committees and check that an attestation comes from a member of the right one, without trusting anybody. Our two designs reuse this pattern of a deterministic function of public randomness that any node can recompute. SOAPS uses it to elect the validator that computes the placement of state on shards, and the witnesses that check it; PANDAS uses it to decide which node is responsible for which part of the data.

An attestation carries three votes. The *head vote* names the block that the validator sees as the head of the chain, and feeds the LMD-GHOST fork-choice rule. The *source* and *target* votes name two checkpoints, and feed the Casper FFG finality mechanism [Sch+22]. Attestations use BLS signatures [BLS01], which can be aggregated [Bon+03]: the signatures of all committee members who cast the same vote collapse into one, with a bit field recording who signed. Without aggregation, about a million signatures per

epoch would have to go into blocks, more than the bandwidth of nodes can carry.

1.3.4 Fork-Choice Rule

The consensus protocol of Ethereum, Gasper, combines two mechanisms [Sch+22; NTT21]. LMD-GHOST, a fork-choice rule, selects the head of the chain and lets it grow even when many validators are offline. Casper FFG, a finality mechanism, runs on top of this chain and finalizes some of its blocks. This is the ebb-and-flow structure of Section 1.2: the chain selected by LMD-GHOST stays available, while the finalized prefix stays safe under network partitions [NTT21].

LMD-GHOST adapts GHOST (Section 1.2.2) to votes. Starting from the latest justified checkpoint, it follows at each fork the branch whose subtree carries the most weight, counting only the latest attestation of each validator (hence *latest message driven*).

Casper FFG reasons in epochs, and the checkpoint of an epoch is the block at its first slot. Once validators holding at least two thirds of the stake attest with the same source and target, the target checkpoint is *justified*. A justified checkpoint is *finalized* when the checkpoint of the next epoch is justified with it as source. Under normal conditions, a block is final after two to three epochs, about 13 to 19 minutes. Reverting a finalized block would mean finalizing two conflicting checkpoints. That takes at least a third of the stake signing contradictory votes, which can be proven and slashed (Section 1.3.5).

The fork-choice rule has been attacked in several ways. In a balancing attack, a few Byzantine validators time the release of their votes so that honest validators stay split between two branches [NTT21]. In an ex-ante reorganization, the attacker withholds a block and its votes, and releases them later to outweigh the next honest block [Sch+22]. The protocol now counters both with *proposer boost*. A block received on time, before the 4 s mark, gets extra

weight in the fork choice until the end of the slot, equal to 40% of the weight of a committee.

Validators should vote only for blocks that are valid and whose data is *available*, meaning that anyone can download it. With Data Availability Sampling (Section 2.4.4), they check availability by downloading a few random samples of the data rather than all of it. The *tight* fork-choice rule has them run this check before attesting, so a block whose data cannot be retrieved never becomes the head, however valid it is otherwise. The *trailing* rule runs the check later. Validators get more time, but the guarantee is weaker, since others may build on a block before its data has been checked [DZS24]. PANDAS targets the tight rule, which is why sampling must finish within the first 4 seconds of the slot [Pig+25].

1.3.5 Rewards and Safeguards

Validators are paid for performing their duties correctly and on time, and lose a little when they miss them. The reward for an attestation depends on which of its three votes are correct and on how quickly it was included in a block. Proposers earn rewards for including attestations and evidence of misbehavior, and sync committee members for signing headers.

A validator is *slashed* for signing messages that contradict each other: two different blocks for the same slot, two attestations with the same target epoch but different content, or an attestation whose source and target surround those of another of its own [Sch+22]. The proposer that includes the resulting proof (Section 1.2.3) receives a reward. The slashed validator immediately loses 1/4096 of its effective balance, is forced to exit, and can withdraw what remains only after 8192 epochs, about 36 days. Halfway through this period, it pays a *correlation penalty*: the fraction of the total stake slashed in the

same window, multiplied by three, is taken from its balance, up to the whole of it. A one-off fault, usually a misconfiguration, costs little. A coordinated attack by a third of the stake costs the attackers all of it.

Another safeguard covers the case where finality stops. With more than a third of the stake offline, no checkpoint can gather two thirds of the votes, and the chain stops finalizing. After four epochs without finality, the *inactivity leak* kicks in. Validators that do not participate lose balance at an increasing rate, until those still active again hold two thirds of the stake and finality resumes. The leak has to be designed with care: Pavloff et al. show that Byzantine validators can use it to drain the stake of honest validators and raise their own relative weight [PAT24].

Ethereum, then, does not prevent misbehavior. It punishes it whenever it can be proven and attributed to a validator. SOAPS relies on these same mechanisms (election, stake, and slashing) instead of adding a consensus pro-

tocol of its own. Every deviation from SOAPS must therefore be detectable and attributable to a specific validator, which is the purpose of its challenge mechanism.

Finally, the block of a slot is often not built by its proposer. The order of transactions in a block can be profitable, for example when one's own transaction is placed just before or after a large trade. This profit is called *maximal extractable value* (MEV) [Dai+20]. Capturing it takes specialized infrastructure, and Ethereum has adopted, outside the protocol, a form of *proposer-builder separation* (PBS). *Builders* assemble blocks and bid for them to be proposed; *relays* pass the highest bid to the proposer, which signs the header without seeing the content. A handful of builders produce most blocks [Hei+23]. For PANDAS, this means that the data to distribute and sample starts at a few well-connected, well-provisioned builders, while most of the nodes sampling it run on modest hardware.

1.4 Transaction and State Models

The state machine view of Section 1.1 says nothing about what the state contains, or how a transaction designates the part of it that it reads and modifies. That is the role of the state model. It defines which data a transaction accesses, and so when two transactions conflict, which in turn governs whether they can run in parallel (Section 2.2.1) and how the state can be split between shards (Section 2.3.3). Bitcoin introduced the *unspent transaction output* (UTXO) model [Nak08], in which the state is a set of coins. Ethereum uses the *account-based* model [Woo14], in which it is a set of accounts with balances and storage. We work with accounts throughout, but start with UTXOs, because the contrast explains several difficulties that Parts II and III deal with.

1.4.1 UTXO Model

In the UTXO model, the state is the set of transaction outputs not yet spent. An output holds an amount and a spending condition, usually a signature from the key of its owner. A transaction consumes existing outputs, its *inputs*, and creates new ones. Each input names the output it spends, by the identifier of the transaction that created it and its index there, and supplies the data that satisfies the spending condition. Outputs cannot exceed inputs in total, and the difference goes to the block producer as a fee (Figure 1.2). An output can be spent only once. Two transactions spending the same output form a *double spend*, and at most one of them makes it into the ledger.

Because inputs are listed in the transaction, the state a transaction depends on is known before it runs. Two transactions conflict only if they spend the same output, or if one spends an output that the other creates. Transactions with no outputs in common can be validated in parallel. A sharded

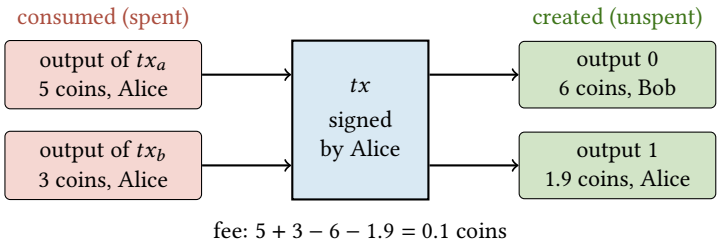


Figure 1.2: A transaction in the UTXO model. It consumes two outputs owned by Alice, which are removed from the set of unspent outputs, and creates two new ones: a payment to Bob and the change returned to Alice. The difference between inputs and outputs is the fee.

design can place each output on a shard, say by the identifier of the transaction that created it, and know in advance which shards a transaction touches. Several of the first sharded blockchains were built on UTXOs for this reason [Kok+18; ZMR18], and transaction placement methods have been proposed for the model [Ngu+19].

The catch is that outputs are consumed whole and hold no persistent data. Applications with shared, long-lived state, such as a token or an exchange used by many users, are hard to express. Extended UTXO models attach data and more expressive scripts to outputs while keeping explicit inputs, but the application still has to split its state into outputs that are consumed and recre-

ated on every update [Cha+20]. The workloads we study and the systems we target use accounts, so we leave UTXOs aside from here on.

1.4.2 Account-Based Model

In the account-based model, the state maps addresses to accounts. Ethereum has two kinds of accounts. *Externally owned accounts* (EOA) are controlled by a private key and hold a balance and a *nonce*, the number of transactions the account has sent. Every transaction carries the nonce of its sender. This forces the transactions of an account to execute in order, and keeps a signed transaction from being replayed. *Smart contract accounts* (SCA) are created by deploying code. Besides a balance and a nonce, they hold that code and a persistent *storage*: a mapping from 256-bit keys, called slots, to 256-bit values, which only the code of the contract can modify (Section 1.5). A token contract typically keeps the balance of each holder in its own storage slot.

Transactions are sent by EOAs. A transaction transfers currency to an-

other account, deploys a new contract, or calls an existing one with some input data. The called contract may call others, which may call others again, so one transaction can read and modify many accounts and storage slots. Only the original transaction appears in the block. The *internal transactions* it triggers can only be observed by executing it (Section 1.5.1) [PKR23]. Activity is also very uneven: a small number of accounts, such as exchanges, popular token contracts and decentralized finance protocols, take part in a large share of all transactions [GDW19].

The key difference from UTXOs is that the state a transaction will access is not known in advance. It depends on the code of the contracts involved and on the current contents of their storage, which may change between the moment the transaction is signed and the moment it runs. The same call can touch different slots, or even different contracts, depending on what executed before it. Whether two transactions conflict, i.e., whether one writes data

that the other reads or writes, is generally known only after both have run. Ethereum does let a transaction declare an *access list* of the accounts and slots it plans to use [BS20]. The list is optional, though, and the transaction may still touch state it did not list. This makes parallel execution (Section 2.2.1) and state sharding (Section 2.3.3) harder with accounts than with UTXOs. Part II measures how real transactions spread their accesses over time and across one another, and the same uncertainty complicates the placement of accounts in SOAPS (Part III).

Executing a transaction costs *gas*. Each operation of the execution environment has a fixed gas cost reflecting the resources it uses, and storage reads and writes are among the most expensive. The sender sets a maximum amount of gas and pays for the gas actually used. Since the London upgrade, the gas price has two parts [But+19; Rou24]. A *base fee*, set by the protocol, rises when blocks are fuller than a target, falls when they are emptier, and

is burned. A *priority fee*, chosen by the sender, goes to the proposer. When demand is high, it decides which transactions get in first, and so how long a transaction waits to be included [Pac+23]. Each block also has a *gas limit* that bounds its total computation, and this limit is, in effect, the throughput of the chain. It is kept low enough for nodes on modest hardware to execute a block within a slot, a direct consequence of full replication (Section 1.1).

1.5 Smart Contracts and Execution Environments

A *smart contract* is a program stored in the ledger and executed by every full node whenever a transaction calls it. Its state, such as balances or open positions, is part of the replicated state, and only the code of the contract can modify it. All nodes run the same code on the same state and get the same result, so nobody has to be trusted to execute the contract correctly.

Much of the activity on Ethereum goes through a few kinds of contracts. Token contracts record who owns which units of a fungible asset (the ERC-20

interface) or which non-fungible items (ERC-721). Around them sit decentralized exchanges, which let users swap one token for another against liquidity pools held by a contract, lending protocols, which lend tokens against collateral, and bridges, which lock assets on one chain to release equivalent assets on another. In the workload analyzed in Part II, transactions involving contracts took over as time went on, and simple transfers, which made up more than 80% of early transactions, fell to around 20% by the Merge [PKR23].

Contracts call each other, and applications are often assembled from existing contracts. A swap on a decentralized exchange may go through a router contract, then through one or more pools, each calling the token contracts of the assets it holds, so a single transaction can end up involving tens of contracts. Even the simplified two-step swap of Figure 1.3 involves six.

These calls are what create dependencies between accounts in the account-based model (Section 1.4.2). They also explain why some contracts, widely

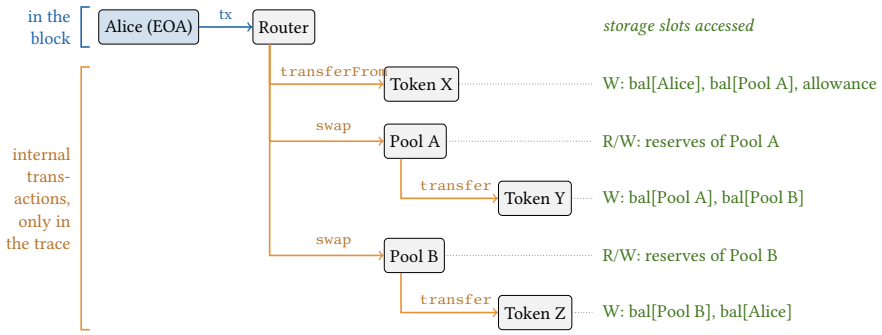


Figure 1.3: Calls made by a transaction in which Alice swaps token X for token Z through two exchange pools (X to Y in Pool A, then Y to Z in Pool B). Only the transaction from Alice to the router is recorded in the block. The internal transactions, and the storage slots that each contract reads (R) and writes (W), are only visible by executing the transaction.

used tokens and exchange pools in particular, are accessed by a large fraction of all transactions. Such contracts turn into contention points as soon as transactions run in parallel or the state is split between shards.

1.5.1 The Ethereum Virtual Machine

Ethereum contracts run on the Ethereum Virtual Machine (EVM) [Woo14], a stack machine that operates on 256-bit words. During a call, a contract sees a stack, a temporary *memory* discarded when the call ends, the input data of the call, and its persistent *storage* (Section 1.4.2). Every instruction

costs gas, and execution aborts with an error when the gas of the transaction runs out, so every execution terminates. Storage accesses are by far the most expensive instructions. Reading a slot for the first time in a transaction costs hundreds of times more than an arithmetic operation, and writing a slot that was previously zero costs more still, because each such operation reads or updates the state database of every node.

A contract calls another with CALL, which runs the code of the callee on the storage of the callee. STATICCALL does the same but forbids any change to the state. DELEGATECALL runs the code of the callee on the storage of the caller, which is how libraries and upgradable contracts work. When a call fails, its changes to the state are reverted, and the caller is free to continue and handle the failure. Transactions in a block run one after the other, in the order chosen by the block producer, each seeing the state left by the previous one.

The state is stored in Merkle Patricia Tries, which combine the Merkle trees of Section 1.1 with a prefix tree (Patricia trie), so that each key determines a unique path from the root to its value. There are two levels. The *world-state trie* maps the hash of each address to its account: nonce, balance, code hash, and storage root. Each contract has its own *storage trie*, which maps the hash of each slot to its value. The root of the world-state trie is the state root found in each block header. Reading a value means walking from the root down to a leaf, which may take several disk accesses. Because keys are hashed, entries that belong together in the application, two slots of the same contract for instance, land in unrelated parts of the trie and of the disk. State access is one of the main costs of executing a block, and clients cut it down with caches and flat copies of the latest state. Those caches only pay off if transactions tend to access state that was accessed recently, a property called temporal locality (Section 2.2).

The block alone does not reveal the read and write sets of a transaction, since internal transactions are not recorded in it (Section 1.4.2). To get them, we replay the whole chain with the Go Ethereum (geth) client and record, for every transaction, the accounts it reads and writes, including those touched by its internal transactions (Figure 1.3). Replaying the chain up to the Merge takes about three days and yields a dataset of about 2 TB; Part II describes the tool we built to process it [PKR23].

Sequential execution over a single global state, together with a gas limit that keeps execution within reach of modest nodes (Section 1.4.2), puts a hard cap on the throughput of Ethereum. In October 2026, the gas limit is 60 million gas per block [Ethd]. **Matthieu** *CHECK: gas limit and average tx/s at submission; update the derived 2,850 tx/block and 240 tx/s.* The cheapest transaction, a plain transfer, costs 21,000 gas [Woo14], so a 12-second slot holds at most about 2,850 transactions, or under 240 tx/s. Real transactions use far more gas, and

the chain processes around 20 tx/s on average [Ethe].

1.5.2 Alternative Execution Environments

Many other blockchains reuse the EVM, as do most layer-2 protocols (Section 2.4). EVM compatibility lets them run existing contracts and reuse the same development tools, and their workloads tend to look like that of Ethereum, because they host the same kinds of applications. Chains with other virtual machines, for example based on WebAssembly, usually keep the same model of contracts executed sequentially over shared state.

Another line of work changes the execution model itself to allow parallel execution. Dickerson et al. proposed that block producers execute contract calls speculatively in parallel and publish the resulting schedule, so that validators can replay the block in parallel and deterministically [Dic+17]. Several recent blockchains were designed for parallel execution from the start. Solana has each transaction declare the accounts it will read and write, and

runs in parallel the transactions whose accounts do not overlap [Yak19]. Sui models the state as objects, and separates objects owned by a single user from shared ones. Transactions that use only owned objects need no ordering relative to others and skip consensus; only those on shared objects go through it [Bla+24]. Aptos uses Block-STM, which runs the transactions of a block optimistically in parallel, detects conflicts at runtime, and re-executes the transactions involved, giving the same result as a sequential execution in block order [Gel+23].

These designs take one of two approaches. Either transactions declare their dependencies up front, which comes naturally with UTXOs but constrains how contracts are written with accounts, or the system executes optimistically and bets on conflicts being rare. Either way, the gain depends on the workload, since transactions that access the same accounts still have to run one after the other. Section 2.2 returns to these techniques.

The Scalability Problem | 2

Chapter 1 ended on a limit built into the classical design: every full node stores the whole state and executes every transaction, so adding nodes adds no capacity. This chapter looks at what that limit costs and at the main ways around it. For each approach, it identifies what is still missing. These gaps are where the contributions of this thesis come in.

Section 2.1 measures the problem in throughput, latency, and the cost of running a full node. Raising throughput by making blocks bigger or more frequent drives that cost up, and with it the risk of fewer, larger participants. The general answer is to make sure no single node handles every transaction or every piece of data. On the layer-1 chain this means sharding computation, and for the data published by layer-2 protocols it means sharding data.

Before turning to protocol changes, Section 2.2 covers optimizations that leave the protocol alone but rely on the workload behaving well. Parallel execution assumes that concurrent transactions rarely conflict. Light nodes, stateless clients and snapshot-based bootstrap assume that recently used state is reused. These two assumptions, spatial and temporal locality, are the ones Part II tests on Ethereum's history.

Section 2.3 turns to on-chain scaling by sharding. It considers in turn how nodes, transactions and state are assigned to shards, and why transactions that span several shards are expensive. It then reviews existing sharded designs and the work on placing state to reduce cross-shard transactions, which Table 2.1 compares. Hash-based placement ignores the workload and makes most transactions cross-shard. Optimized placements assign every account explicitly, which is too much metadata at Ethereum's scale. Most also leave open who computes the placement and how other nodes can check it. Part III

addresses these issues.

Section 2.4 covers off-chain scaling with layer-2 protocols: rollups, state channels and sidechains. Rollups have become the main path for Ethereum, but they move the pressure onto layer 1, which must make their data available. The section follows this need from calldata to the blobs of EIP-4844, then to data availability sampling and the Danksharding target. The open problem is to disseminate and sample that data within the four seconds that Ethereum's consensus allows, at full scale and despite faulty nodes, and Part IV addresses it.

2.1 Throughput, Latency, and Full-Node Cost Bottlenecks

Full replication (Section 1.1) makes a blockchain secure without trusted parties, but it costs performance in three ways. Throughput is limited by what a single node can process. Latency is limited by how quickly blocks propagate and consensus concludes. And whatever the system demands of one node, it

demands of every node. This section looks at each of these limits and at why they cannot be relaxed independently.

Throughput. Every full node executes every transaction, so the chain as a whole processes no more than one node can. Bitcoin reaches at most about 7 transactions per second. Visa, as a reference payment processor, handles about 2,000 per second on average, with peaks at 56,000 [Cro+16]. Ethereum does somewhat better but stays in the same order of magnitude. Its gas limit allows at most a few hundred simple transfers per second, and the chain processes around 20 transactions per second on average (Section 1.5.1).

The obvious fix is to raise the parameters: bigger blocks, a higher gas limit, or shorter intervals between blocks. Each of these makes blocks take longer to propagate relative to the interval between them. That produces more forks, wastes honest work, and makes attacks such as double-spending and selfish mining cheaper [Ger+16]. Croman et al. measured this trade-off on Bitcoin's

network in 2016. To keep at least 90% of nodes able to follow the chain, blocks could grow to at most 4 MB at a 10-minute interval, about 27 transactions per second, and the interval could not go below 12 s without wasting network bandwidth [Cro+16]. Tuning the parameters gains a constant factor at best, not the orders of magnitude separating blockchains from payment networks.

Latency. A transaction is first *included* in a block, then becomes *final*, meaning it can no longer be reverted. Both steps are bounded by the protocol. On Ethereum, a block is produced every 12 s slot, but inclusion can take several slots when the network is congested or the fee offered is low [Pac+23]. **Matthieu** *CHECK: that Pacheco et al. actually support this (fees/congestion delaying inclusion); I did not take a figure from them.* Finality comes from Casper FFG and takes two to three epochs, about 13 to 19 minutes (Section 1.3.4). Bitcoin offers only probabilistic finality, and users wait for several blocks of ten minutes each before treating a payment as settled [Nak08]. **Matthieu** *CHECK:*

the whitepaper gives the catch-up probability, not the "wait several blocks" convention; consider a better source or rephrase. The same propagation and consensus constraints that cap throughput make these delays hard to shorten.

Full-node cost. Running a full node costs storage, bandwidth and computation, and these costs are what keep participation open. Ethereum's hardware recommendations for a full node are a 4 TB NVMe disk, 32 GB of memory, a four-core CPU, and 50 Mbps download and 15 Mbps upload [Jay+25]. These are within reach of an individual, and stay so only if the load does not grow. Storage grows with the state and the history. Ethereum's state holds about 400 million accounts (Part III). **Matthieu** *CHECK: source for 400M accounts (SOAPS states it without citation); make sure Part III gives the figure and its date.* The history up to the Merge alone has 1.7 billion transactions touching 180 million accounts, and replaying it takes about three days and produces a database of about 2 TB [PKR23]. **Matthieu** *CHECK: the BRAINS paper gives 1.7 billion transac-*

tions in Sec. II-B, 1.9 billion in Sec. III-C and 1.3 billion in Sec. IV (different counting); pick one definition and use it consistently in Part II. Every increase in throughput adds to these costs. As they rise, fewer participants can afford a node, and the network concentrates in the hands of those who can, which is precisely what decentralization is supposed to avoid [Ger+16; Cro+16].

Data bandwidth. For the data that layer-2 protocols publish on layer 1 (Section 2.4), bandwidth runs out first. With EIP-4844, every node downloads every blob, which is why only a few blobs of 128 KB fit in each block [But+22] (Section 2.4.4). The Danksharding target of 32 MB per block [Eth24] cannot work this way. To attest within the first 4 s of a slot, a node would need to receive the raw data at about 64 Mbps, or 280 Mbps for the 140 MB erasure-coded version. Both exceed the recommended 50 Mbps download [Jay+25].

Matthieu *CHECK: own back-of-the-envelope numbers (32 MB and 140 MB in 4 s, MB = 10^6 B, no protocol overhead). Decide whether to present them as such.*

One answer: sharding. These limits are often summarized as the *scalability trilemma*: a blockchain cannot be simultaneously scalable, secure and decentralized. We use it only as a framing. The constraint behind it is concrete: as long as every node handles every transaction and every byte of data, capacity is bounded by what one modest node can do. Both layers need more capacity. Layer 1 must execute more transactions, and must also host the data of a growing number of layer-2 protocols. The general way out is *sharding*, in which no single node handles everything. Sharding computation means that each node executes only part of the transactions, which Part III addresses. Sharding data means that each node downloads and stores only part of the published data, which Part IV addresses. The rest of this chapter reviews the approaches that lead to these two forms of sharding, and the alternatives that avoid them.

2.2 Exploiting Workload Properties

Before changing the protocol, one can make each node do its work faster or with fewer resources. Several optimizations do this without touching consensus or the data model. They rely instead on the transaction workload behaving well, and the property they assume is some form of *locality*, the same notion that underlies caches in computer systems. A program does not touch its whole address space at once. Over a short period, it uses a small *working set*, and a system that keeps that set close at hand runs almost as fast as if everything were [Den68]. Two kinds of locality matter for blockchains. Part II gives their formal definitions.

Temporal locality means that state used recently is likely to be used again soon. If it holds, a node can keep the recently used part of the state in memory, or even keep only that part, and fetch or prove the rest on demand. Light nodes, stateless clients and snapshot-based bootstrap all rely on it (Sec-

tion 2.2.2).

Spatial locality means that transactions issued around the same time fall into groups that touch disjoint sets of accounts. Its consequence is *disjoint access parallelism*: groups that share no state can be processed independently, by different cores of one node (Section 2.2.1) or by different shards (Section 2.3).

Many designs assume these properties without measuring them on a real workload [PKR23]. Part II measures them on the full history of Ethereum. Its main finding is that the workload generally shows good locality of both kinds, but a small minority of accounts is responsible for most of the behavior that defeats caching or parallelism. These two minorities are largely distinct. In the pre-Merge history, about 3.2 million accounts stand out for poor temporal locality and as many for poor spatial locality, but only about 0.6 million belong to both sets [PKR23]. **Matthieu** *CHECK: numbers from Table I of the BRAINS paper (pre-Merge data). Update if Part II is redone on recent data.* Each property therefore

calls for its own handling. Part III builds on the minority that hurts spatial locality: it places a small set of high-impact accounts explicitly and leaves all other accounts to the default rule.

2.2.1 Parallel Transaction Execution

Ethereum executes the transactions of a block one after the other (Section 1.5.1). Parallel execution runs transactions that do not conflict at the same time. The result must stay the same as if the block had been executed sequentially in its order: otherwise, nodes executing the block differently would end up with different states. In the account-based model, which transactions conflict is generally known only after they have run (Section 1.4.2). Proposals differ mainly in how they get around this. Section 1.5.2 introduced some of the resulting designs; here we compare them by how they deal with conflicts.

The first approach is to have transactions *declare* the state they access. Solana requires every transaction to list the accounts it reads and writes, and

schedules in parallel the transactions whose lists do not overlap [Yak19]. The schedule is then exact, but the burden falls on users and developers. A transaction that declares too much loses parallelism, and one that declares too little cannot access the state it needs. Ethereum's access lists [BS20] are optional and not enforced, so they cannot serve this purpose as they stand.

The second approach is *optimistic*. Block-STM, used by Aptos, runs the transactions of a block in parallel as if they did not conflict, records what each one reads and writes, and re-executes those that read a value a preceding transaction later changed [Gel+23]. OptSmart applies software transactional memory in the same spirit [Anj+22]. **Matthieu** *CHECK: description of OptSmart (STM-based execution, conflict graph shared with validators) against the paper.* These designs require nothing from users. Their performance, however, depends directly on the workload: when many transactions touch the same accounts, re-executions pile up, and the speedup shrinks toward that

of sequential execution [Gel+23]. **Matthieu** *CHECK: Block-STM behavior under high contention against the paper's evaluation.* Forerunner takes a different angle and speculatively executes pending transactions before the block that contains them arrives, so that their results can be reused once the actual order is known [Che+21]. **Matthieu** *CHECK: description of Forerunner.*

The third approach shares the cost of discovering conflicts. In the design of Dickerson et al., the block producer executes the block speculatively and publishes the schedule it found, which validators then replay in parallel and deterministically [Dic+17]. OptSmart likewise lets the producer ship the conflict graph of the block alongside it [Anj+22]. ParBlockchain, for permissioned blockchains, orders transactions first and builds a dependency graph that lets executors process independent transactions concurrently [AAE19].

Matthieu *CHECK: description of ParBlockchain (order-execute, dependency graph).*

All three approaches gain only as much as the workload allows. Transac-

tions that touch the same account must still run one after the other, so a few very popular contracts can serialize a large part of a block. How much parallelism the Ethereum workload actually offers is a question of spatial locality, and Part II measures it. This thesis does not contribute to parallel execution itself. We return to it only as a perspective, in light of the results of Part II.

2.2.2 Light Nodes and Stateless Clients

The other way to lower the cost of a node is to keep less of the state. At the extreme, a light client stores only block headers and checks the values it cares about, such as its own balance, using Merkle proofs against the state root in a header [CBC22]. In Ethereum, it follows the headers signed by the sync committee (Section 1.3.2). The price is trust. A light client executes nothing, so it accepts whatever block the majority of the committee signs, and on its own it can detect neither an invalid state transition nor data that was never published [CBC22]. Fraud proofs and data availability sampling

were proposed to close this gap: a full node that finds an invalid block can prove it to light clients, provided the data needed to do so is available [Al-+21]. The second condition is the data availability problem that Section 2.4.4 returns to.

Between light clients and full nodes, several designs keep part of the state. Subset nodes store and track only the part of the state that matters to their owner [PCG21]. In CoVer, light nodes collectively verify blocks and keep the data available between them, without relying on full nodes [CKR20].

Matthieu *CHECK: description of CoVer and of subset nodes.* A related problem is bootstrapping a new full node. Instead of replaying the history from genesis, it can start from a recent snapshot of the state and then follow the chain. Ethanos periodically sweeps accounts that have been inactive, so that a new node only needs the recently active ones, and old accounts must be restored with a proof when they are used again [Kim+21]. **Matthieu** *CHECK: description*

of Ethanos (sweeping inactive accounts, restore with proof). Fynn et al. propose a fast and robust protocol for synchronizing a node's state from its peers [Fyn+22].

Matthieu *CHECK: bib entry fynn2022robust: author list not verified (the BRAINS paper credits "Robert Fynn"), and description of the paper.* These designs pay off only if the state that is recently used, or that changes between two snapshots, is a small part of the whole.

Stateless clients take the idea furthest. A stateless node keeps no state at all. Each block comes with a *witness*, the Merkle proofs for every piece of state the block reads or writes, so that any node can execute the block against the state root alone. Witnesses cost bandwidth, and their size depends on the structure of the state tree. Ethereum has therefore considered replacing its Merkle Patricia trie with trees that give shorter proofs, such as Verkle or binary trees. **TODO** *cite (Verkle trees / EIP for the binary state tree)* Nodes that cache recently used state can skip the corresponding parts of the witness.

The benefit of all these designs depends on how often recently used state is reused, and on how much of the state a node actually needs at a given time. That is temporal locality, which Part II measures.

2.3 On-Chain Scaling: Sharding

Sharding applies to blockchains the idea that distributed databases use to scale: partition the work. The nodes, the state and the processing of transactions are split across several *shards* that run in parallel. Each shard has its own committee of nodes, which runs consensus on the blocks of that shard only [Wan+19; Al +24]. A coordinating chain, the *beacon chain* in Ethereum's terminology, keeps the list of nodes and a source of randomness. It assigns nodes to shards and records periodic checkpoints of every shard, which tie the shards into a single system. Ethereum's beacon chain was designed for this role (Section 1.3). If shards rarely need each other, a system with k shards processes up to k times more transactions than a single chain, while each node

still handles the load of one shard.

The price is security. Taking over a whole blockchain requires controlling a large share of all nodes or all stake. Taking over one shard only requires controlling enough of its committee, which is much smaller. Committees are therefore formed at random, so that an attacker cannot choose where its nodes go. The probability that a committee is compromised then follows from sampling without replacement. Consider N nodes, a fraction f of which are malicious, and a committee of m nodes drawn uniformly at random. The number X of malicious members follows a hypergeometric distribution, and a BFT committee fails when $X \geq m/3$:

$$\Pr\left[X \geq \lceil m/3 \rceil\right] = \sum_{x=\lceil m/3 \rceil}^{m} \frac{\binom{fN}{x}\binom{(1-f)N}{m-x}}{\binom{N}{m}}.$$

This probability falls quickly with m. With $N = 10{,}000$ and $f = 1/4$, it is about 3×10^{-2} for committees of 100 nodes, 6×10^{-5} for 400 nodes, and

10^{-6} for 600 nodes. **Matthieu** *CHECK: own computation (exact hypergeometric tail, Python); decide whether to keep these numbers or cite the equivalent figures of OmniLedger/RapidChain.* Sharded designs pick committees large enough for this probability to be negligible over the lifetime of the system. This bounds the number of shards that a given number of nodes can support. Ethereum's committees, with at least 128 members, follow the same reasoning (Section 1.3.3). Random assignment alone does not stop an *adaptive* adversary, which corrupts nodes after seeing where they were assigned. Committees are therefore reshuffled regularly. Avarikioti et al. formalize the properties that a robust sharded blockchain must guarantee, including against such adversaries, and show which design elements they require [Ava+23].

Following Wang et al. [Wan+19] and Al Barat et al. [Al +24], the rest of this section separates three aspects of sharding. *Network sharding* assigns nodes to shards (Section 2.3.1). *Transaction sharding* assigns transactions

to shards (Section 2.3.2). *State sharding* assigns the state, in our case accounts, to shards, and determines which transactions span several of them (Section 2.3.3). For an account-based blockchain, the questions that matter are therefore where nodes, state and transactions are *placed*, and how transactions that cross shards are *handled*. Section 2.3.4 then reviews existing designs and the work on optimizing state placement.

2.3.1 Network Sharding

Network sharding decides which nodes serve which shard. A node then takes part in consensus, and in the dissemination of blocks and transactions, for its own shard only, so its load no longer grows with the total activity of the system. Two threats shape how nodes are assigned.

The first is the Sybil attack, in which an attacker creates many identities to fill a committee [Dou02]. Identities must therefore cost something. Elastico requires each node to solve a proof-of-work puzzle before every epoch, and

the solution also determines the committee it joins [Luu+16]. Proof-of-stake designs, Ethereum included, weight identities by stake instead (Section 1.2.3).

The second is the attacker who chooses, or predicts, where its nodes end up. Assignment must then be random, unpredictable before it happens, and verifiable by everybody after. OmniLedger generates the randomness with RandHound, a distributed protocol for unbiasable public randomness [Kok+18]. RapidChain uses the Cuckoo rule when nodes join and leave, which keeps the malicious nodes spread across committees even when the attacker controls the order of arrivals [ZMR18]. Ethereum assigns validators to committees with a shuffle seeded by the RANDAO value (Section 1.3.3), whose last contributors can bias it slightly by withholding their block [Nag+25]. **Matthieu**

CHECK: that Nagy et al. support this wording (their attack uses forking; withholding is the classic bias).

Reshuffling has a cost. A node that moves to a new shard must obtain the

current state of that shard before it can validate its blocks. Frequent reshuffling leaves an adaptive attacker little time to corrupt a committee, but makes nodes download state over and over. Designs therefore reshuffle once per epoch, or move only part of each committee at a time [Kok+18]. **Matthieu** *CHECK: that OmniLedger swaps validators gradually (bounded fraction per epoch); cite the exact mechanism or drop.*

2.3.2 Transaction Sharding

Transaction sharding decides which shard processes a transaction. Each shard keeps its own pool of pending transactions, and its committee includes them in its blocks. The most common rule sends a transaction to the shard of its sender, or of the account it calls first [Kok+18; ZMR18]. **Matthieu** *CHECK: OmniLedger and RapidChain are UTXO-based (transactions go to input/output shards); find an account-based source for the sender-shard rule or reword.* The rule is simple, known to everyone, and requires no information beyond the address of the

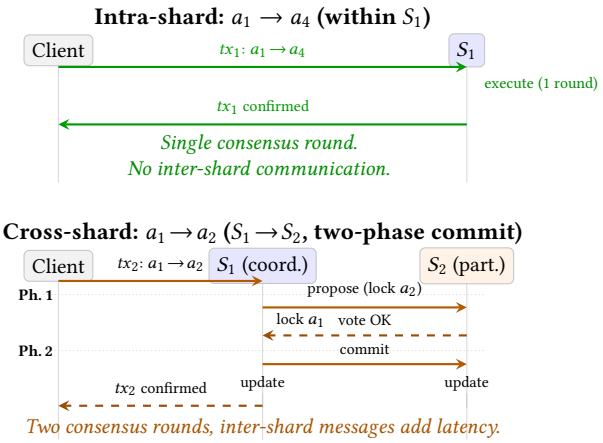


Figure 2.1: Intra-shard vs. cross-shard transactions. (top) An *intra-shard* transaction ($a_1 \to a_4$, both in S_1) completes in a single consensus round. (bottom) A *cross-shard* transaction ($a_1 \to a_2$ across S_1 and S_2) requires two-phase commit: Phase 1 locks state and collects votes; Phase 2 broadcasts commit or abort, adding at least one extra consensus round.

sender. Other routing rules have been proposed, for example to balance load between shards, but they remain uncommon [LNS25].

Where a transaction is sent matters only in relation to where its state lives. A transaction is cheap when all the state it accesses sits in the shard that processes it, and expensive otherwise, whatever the routing rule. The decisive question is therefore how the state is placed.

2.3.3 State Sharding

State sharding splits the state itself, so that each shard stores and updates only its own part. This is the form of sharding that reduces the storage and execution load of each node, and the one Part III deals with. A placement of accounts on shards must satisfy two requirements. It must be known to everybody, because nodes and clients must know where an account lives in order to route transactions that use it. And it must balance the load, so that no shard becomes a bottleneck, in storage or in processing.

Most designs meet both requirements with a hash function: an account with address a lives in shard $h(a) \bmod k$ [Al +24]. Chainspace [Al-+18], Monoxide [WW19] and Elastico [Luu+16] place state this way, as did the Ethereum sharding proposals. **Matthieu** *CHECK: Elastico shards transaction processing but, as far as I know, not storage (all nodes keep the full ledger); it may not belong in this list (it comes from TxAllo's list via SOAPS).* **TODO** *cite a primary source*

for Ethereum's hash-based placement (the SOAPS paper cites EIP-4844, which does not cover it) Hash placement needs no metadata, since anybody can compute the shard of any account, and spreads accounts uniformly. But it ignores entirely which accounts are used together.

A transaction is *intra-shard* when all the state it accesses is in one shard. It then executes in one consensus round of that shard, with no communication with the others. It is *cross-shard* when its state spans several shards. The shards involved must then agree to either all apply the transaction or none of them, which requires an atomic commitment protocol [Adh+25]. The classical choice is two-phase commit, illustrated in Figure 2.1. A coordinator shard, typically that of the caller, locks its state and sends a *propose* message to each other shard involved. Each of them locks the state concerned and replies with a vote. If every vote is positive, the coordinator commits in a second phase and all shards apply the update; otherwise, all shards abort and release their

locks. Each phase involves the consensus of the shards involved, so a cross-shard transaction takes at least one more consensus round than an intra-shard one, plus messages between shards.

Several protocols refine this scheme. In OmniLedger's Atomix, the client itself drives the protocol: it collects proofs that its inputs are locked in each input shard, then presents them to commit, or to unlock if one of them failed [Kok+18]. RapidChain splits a cross-shard transaction into sub-transactions that move the inputs into the shard of the output, which then executes the transaction locally [ZMR18]. Monoxide uses relay transactions between shards and gives up on immediate atomicity in favor of eventual atomicity [WW19]. BrokerChain lets broker accounts hold state in several shards, so that some transfers can go through a broker instead of crossing shards [Hua+22]. Zhang et al. survey these and other optimizations [Zha+25]. They reduce the overhead of cross-shard transactions, but none removes it: the extra latency and

messages, the time during which state stays locked, and the transactions that abort all remain. How many transactions are cross-shard is therefore a first-order factor in the performance of a sharded blockchain.

Under hash placement, nearly all of them are. With k shards, two accounts land in the same shard with probability $1/k$, and a transaction touching n distinct accounts is intra-shard with probability $(1/k)^{n-1}$. Since Ethereum transactions often involve several contracts (Section 1.5), very few remain local. On recent Ethereum traces, with 64 shards, 94.6% of transactions would be cross-shard (Part III). **Matthieu** *CHECK: 94.6% comes from the SOAPS paper; keep in sync with the final Part III numbers.* Even an ideal placement cannot bring this to zero. Part II partitions each window of one million transactions of the pre-Merge history with METIS, with complete knowledge of the accounts that the window accesses. About 25% of transactions still cross partitions with 16 partitions, and 30% with 64. This share has grown steadily over

Ethereum's history [PKR23], consistent with the observations of the authors of OptChain [Ngu+19]. **Matthieu** *CHECK: METIS 25–30% figures are pre-Merge (BRAINS paper); update if Part II is redone.* The two figures come from different datasets and should not be compared directly, but together they bound the problem. Hash placement leaves almost every transaction cross-shard, and a placement aware of the workload can do much better, but not perfectly.

2.3.4 Existing Sharded Blockchains and Their Limitations

The first generation of sharded blockchains established the techniques described above. Elastico was the first to shard transaction processing in an open setting, with proof-of-work identities and committees re-formed every epoch [Luu+16]. OmniLedger added secure randomness for committee formation, gradual reconfiguration, and the Atomix protocol for cross-shard transactions [Kok+18]. RapidChain reduced communication within committees and handled the joining and leaving of nodes with the Cuckoo rule [ZMR18].

Chainspace sharded a platform for smart contracts over objects, with a commit protocol across shards [Al-+18]. Monoxide relaxed cross-shard atomicity to scale to many zones [WW19]. **Matthieu** *CHECK: one-line summaries of Elastico, OmniLedger, RapidChain, Chainspace and Monoxide against the papers.* Elastico, OmniLedger and RapidChain target UTXO payments, Chainspace manipulates objects, and Monoxide uses accounts. All of them place state, or transactions, by hash, and so pay for cross-shard transactions on a large share of the workload. **Matthieu** *TODO: consider a small table (node placement, state placement, transaction placement, cross-shard protocol) for these designs; the SOAPS draft had one, check each cell against the papers.*

Some sharded blockchains are in production, such as NEAR and MultiversX. **Matthieu** *TODO: cite NEAR (Nightshade) and MultiversX (Adaptive State Sharding) or drop.* Ethereum, however, abandoned execution sharding. Its earlier roadmap planned shard chains, each with its own state and transactions,

coordinated by the beacon chain. It moved instead to a single execution chain, with rollups for scaling execution and sharding of data only, in the Danksharding design [Fei22; Eth24]. Section 2.4 returns to this choice. The beacon chain kept the role that sharding had given it, and SOAPS builds on that design (Part III).

Optimizing state placement. The first limitation of these designs is that hash placement ignores the workload [Kró+21]. A large body of work places state so as to reduce cross-shard transactions while keeping the load balanced. Fynn and Pedone first raised the question of a globally good partitioning of blockchain state [FP18]. Finding an optimal placement is NP-hard [MR20], so practical approaches model past transactions as an *interaction graph*, whose vertices are accounts and whose edges join accounts used by the same transactions. They then apply graph partitioning or community detection to group accounts that are used together. The idea comes from distributed databases.

Schism partitions tuples with METIS [KK98] on the graph of tuples accessed by the same transactions [Cur+10], and Marlin re-shards Byzantine-resilient databases as their workload shifts [Meh+25].

For account-based blockchains, these proposals fall into three families. *Reactive*, node-initiated approaches let shards migrate accounts on the fly, according to the cross-shard traffic they observe [MR20; Kró+21] or to load imbalance [TDL22]. *Periodic*, global approaches recompute an account-to-shard map at fixed intervals and broadcast it to all nodes. They use community detection algorithms such as Louvain [Blo+08; Zha+22], label propagation [Li+22], or the CPLA algorithm of ShardCutter [Hua+26]. TxAllo refines the global map locally in each shard [ZPY23], and EfShard indexes it with a layered combination of hashing and fine-grained mapping [MW23]. BrokerChain also partitions the interaction graph, and spreads the most active accounts over several shards through its brokers [Hua+22]. *Predictive* ap-

proaches anticipate future interactions, with reinforcement learning (Spring [Li+24], Haider et al. [Hai+25]), or give newly created accounts an informed first placement (TransShard [WG24], Li et al. [LNS25]). At the level of transactions, OptChain chooses the shard of each new transaction to minimize future cross-shard transactions [Ngu+19]. It targets the UTXO model, however, and was evaluated on Bitcoin, so it does not compare directly with the account-based schemes. **Matthieu** *CHECK: that OptChain's evaluation uses Bitcoin data.* The account-based schemes are mostly evaluated on Ethereum traces, and report large reductions of the cross-shard ratio. Zhang et al., for instance, report up to 50% fewer cross-shard transactions with 16 shards while keeping the load balanced [Zha+22]. **TODO** *check this figure in zhang2022community (it comes from commented-out SOAPS draft text)*

The cost of explicit placement. These gains come at a cost that most of this work leaves aside. Every account receives an explicit location, which ev-

ery node must know in order to route transactions. Ethereum has about 400 million accounts. Even with a single byte per shard identifier, the map takes about 400 MB, which every node must store and keep synchronized each time the placement changes. **Matthieu** *CHECK: source for 400M accounts (SOAPS states it without citation); make sure Part III gives the figure and its date.* Hash placement needs none of this. Only Mizrahi and Rottenstreich bound this state explicitly [MR20], and Haider et al. partially [Hai+25].

Who decides the placement. The last limitation is governance. Most proposals have a single coordinator compute the placement, or do not say who computes it, on what input, or how other nodes can check the result. In an open blockchain, this matters. Cross-shard transactions are slower, so whoever controls placement can give its own accounts, or those of an application it runs, a better placement at the expense of a better global one. We call this *selfish optimization*. It could also choose the shard of a target account, re-

moving one of the conditions that an attacker otherwise has to meet: that the shard whose committee it manages to control is the one that happens to hold the target. A few designs do make placement verifiable. Li et al. [Li+22], ShardCutter [Hua+26] and BrokerChain [Hua+22] run it through consensus, but at the price of having every node, or a whole BFT committee, redo the expensive optimization. **Matthieu** *CHECK: this characterization of Li et al., ShardCutter and BrokerChain comes from the SOAPS intro; confirm against each paper.* EfShard is the only prior design whose placement is both auditable and incentivized [MW23].

Adjusting at runtime. Another line of work leaves the placement mostly alone and reduces the cost of cross-shard activity as it happens. Some designs migrate accounts between shards: Król et al. move accounts towards the shards they interact with [Kró+21], and Huang et al. lock accounts during their migration with a fine-tuned mechanism [HLZ24]. BIND goes further

and keeps processing transactions on an account while it migrates [Qi+26]. Concordia instead has shards cooperate when scheduling transactions, to avoid cross-shard conflicts [Liu+26]. These approaches pay a runtime cost in migration or scheduling overhead, rather than in placement metadata.

Summary. Table 2.1 compares the placement optimizations against six criteria. Two are the optimization targets: *load balancing* keeps storage and processing even across shards, and *cross-shard reduction* cuts the number of cross-shard transactions. The other four concern integration in an open blockchain. *Auditable* means any node can check the placement it is given. *Incentivized* means computing a correct placement is rewarded and deviating penalized. *Adaptive* means the placement follows changes in the workload. *State-bounded* means the placement metadata that every node keeps does not grow with the number of accounts. **Matthieu** *CHECK: ticks copied from the SOAPS table; EfShard year changed to 2023 to match its bib entry (also fix in SOAPS); mixed bold/plain col-*

Protocol	Year	LB	CSR	Auditable	Incentivized	Adaptive	State-bounded
Mizrahi and Rottenstreich [MR20]	2020	✓	✓	✗	✗	✓	✓
Król *et al.* [Kró+21] (Shard Scheduler)	2021	✗	✓	~	~	✓	✗
Toulouse *et al.* [TDL22]	2022	✓	✗	✗	✗	✓	✗
Zhang *et al.* [Zha+22]	2022	✓	✓	✗	✗	✗	✗
Li *et al.* [Li+22]	2022	✓	✓	✗	✗	✓	✗
Zhang *et al.* [ZPY23] (TxAllo)	2023	✓	✓	✗	✗	✓	✗
Mu and Wei [MW23] (EfShard)	2023	✓	✓	✓	✓	✓	✗
Wang *et al.* [WG24] (TransShard)	2024	✓	✓	~	✗	✓	✗
Li *et al.* [Li+24] (Spring)	2024	✓	✓	✗	✗	✓	✗
Haider *et al.* [Hai+25]	2025	✓	✓	✗	✗	✓	~
Li *et al.* [LNS25]	2025	✓	✓	✗	✗	~	✗
Huang *et al.* [Hua+26] (ShardCutter)	2026	✓	✓	✓	✗	✓	✗
SOAPS (Part III)	**2026**	✓	✓	✓	✓	✓	✓

Table 2.1: Prior work on optimizing state placement in account-based sharded blockchains, compared against the criteria of Section 2.3.4. LB and CSR stand for the *load balancing* and *cross-shard reduction* optimization targets; ~ means partial support.

umn headers kept from SOAPS. No prior design meets all four. What is missing is a placement that reduces cross-shard transactions with bounded metadata, that any node can verify, and that is integrated with the consensus of the blockchain and its incentives, without every node redoing the computation. Part III presents SOAPS, which addresses this gap.

2.4 Off-Chain Scaling: Layer 2

Sharding changes the layer-1 chain itself. The alternative is to leave the chain as it is and move transactions elsewhere. *Layer-2* protocols run transactions outside the main chain, the *layer 1*, and use it only to settle the outcome and, depending on the design, to publish the data needed to check that outcome [Gud+20]. Gudgeon et al. classify them into channels, commit-chains and sidechains, and rollups. **Matthieu** *CHECK: that Gudgeon et al. (FC 2020) actually cover rollups; if not, cite a more recent L2 survey (e.g., PANDAS cites Gangwal et al. 2023) for the taxonomy and for the rollup description.* They differ above all in how much of the security of layer 1 they keep [Gud+20]. A useful way to compare them is to ask what an honest user needs in order to recover their funds if the operator of the layer-2 protocol cheats or disappears.

Layer-2 protocols do not make the scaling of layer 1 unnecessary. They move the pressure elsewhere: from executing transactions on layer 1 to pub-

lishing data on it. Ethereum made this explicit in 2020 with its *rollup-centric* roadmap [But20]. Execution scales through rollups, and layer 1 becomes the place where they settle and publish their data. This section presents rollups first, as the dominant design, then state channels and sidechains. It ends with the data availability problem that rollups raise.

2.4.1 Rollups

A rollup is a chain whose blocks are produced off layer 1 but whose state is anchored in it. A *sequencer* receives the transactions of rollup users, orders and executes them, and periodically posts two things to layer 1. The first is the transaction data in compressed batches. The second is a commitment to the resulting rollup state, in the form of a state root [Gud+20]. A *bridge* contract on layer 1 holds the assets that users deposited into the rollup and records the state roots, against which withdrawals are paid. In most rollups today, the sequencer is a single operator. It can delay or censor transactions, but

users can force a transaction into the rollup by submitting it through layer 1.

Optimism illustrates the structure [Opt]. Its sequencer produces rollup blocks. A *batcher* posts the data of these blocks to layer 1, as blobs since EIP-4844 (Section 2.4.4). A *proposer* periodically posts output roots, commitments to the rollup state. Anyone can rebuild the whole rollup chain from the data on layer 1 alone, without trusting the sequencer. **Matthieu** *TODO: add a figure (users, sequencer, batches/blobs on L1, bridge, verifiers rebuilding the state).*

This last property is what the security of a rollup rests on. A state root posted to layer 1 is only a commitment. To check that it is correct, or to know what one owns in order to withdraw, one must be able to recompute the state, and so the data must be *available*: published, and retrievable by anyone who wants it [Al-+21]. Rollups then differ in how layer 1 is convinced that the posted state roots are correct.

2.4.1.1 Optimistic Rollups

An optimistic rollup assumes that posted state roots are correct. A root is accepted unless someone challenges it with a *fraud proof* during a challenge window, about seven days in deployed rollups such as Arbitrum and Optimism [Kal+18; Ethb]. Checking a fraud proof does not require layer 1 to re-execute a whole batch. In Arbitrum, the challenger and the party that posted the root bisect the disputed execution interactively, until they disagree on a single instruction, which the layer-1 contract then executes to settle the dispute [Kal+18]. A single honest participant watching the rollup is enough to stop an invalid state root.

The price is latency. A withdrawal to layer 1 can only be paid once the window has passed, so users wait about a week, or pay a third party to advance them the funds. The scheme also depends entirely on data availability. To build a fraud proof, a challenger must recompute the disputed state from

the batch data. If the sequencer posts a state root but withholds the corresponding data, nobody can prove the root wrong, and an invalid root is accepted at the end of the window.

2.4.1.2 Zero-Knowledge Rollups

A zero-knowledge rollup, or validity rollup, attaches to each batch a succinct proof, a SNARK or a STARK, that the new state root results from correctly executing the batch on the previous one. A contract on layer 1 verifies the proof, and the state root is final as soon as the proof is accepted, with no challenge window [Gud+20; Ethc]. The cost moves to the prover. Generating a proof for EVM execution is expensive, and *zkEVM* designs trade some compatibility with the EVM for faster proving.

A validity proof removes the need for fraud proofs, but not the need for data. The proof shows that the new state is correct, not what it contains. Without the data, users cannot know their balances, build the Merkle

proofs needed to withdraw, or take over if the operator disappears. Optimistic and zero-knowledge rollups alike therefore need their data to be available on layer 1.

2.4.2 State Channels

A state channel lets a fixed group of participants transact among themselves without touching the chain, except to open and close the channel [Gud+20]. The participants lock funds in a contract on layer 1 to open the channel. They then exchange signed updates of the channel state off chain, as often as they like, and close the channel by submitting the final state. If one of them submits an outdated state, the others have a dispute period to submit a more recent signed one, which takes precedence. The Lightning Network applies this to Bitcoin payments, and routes a payment across a path of channels so that two parties without a direct channel can still pay each other [PD16].

Within a channel, transactions are instant and cost nothing on chain. But

funds stay locked for the lifetime of the channel, participants must stay online, or delegate to a watchtower, to react to disputes, and the set of participants is fixed in advance. Channels suit repeated payments between known parties, not general applications with many users interacting with shared contracts.

2.4.3 Sidechains

A sidechain is a separate blockchain with its own consensus, connected to layer 1 by a bridge, Polygon PoS being an example [Gud+20]. Assets move between the two chains through a two-way peg. They are locked on layer 1 so that an equivalent amount can be minted on the sidechain, and burned on the sidechain to be released on layer 1. A sidechain can have its own parameters, with faster blocks, higher throughput or cheaper transactions, since it is not bound by the constraints of layer 1.

It does not inherit the security of layer 1, however. Funds on a sidechain are only as safe as its validators, and funds in the bridge only as safe as the

committee or contract that controls it. Bridges have suffered some of the largest thefts in the history of blockchains. **Matthieu** *TODO: cite a bridge-hack study or SoK (e.g., Ronin, Wormhole) for this claim.* Commit-chains such as Plasma sit between sidechains and rollups. They post only commitments to layer 1 and keep the data off chain, so when an operator withholds data, users must exit in time on their own [Gud+20].

The comparison explains the direction Ethereum has taken. A rollup inherits the security of layer 1 because everything needed to check it and to exit from it is on layer 1: the state roots, the proofs or the possibility of fraud proofs, and the data. Sidechains and commit-chains give up part of this, and channels are not general-purpose. Rollups thus became the main path for scaling execution on Ethereum [But20]. That choice turns the data that rollups publish into the resource that layer 1 must scale.

2.4.4 Existing Layer 2 and Data Availability Solutions and Their Limitations

The data availability problem. How can a node be sure that data published with a block can be retrieved, without downloading all of it [Al-+21]? The question is harder than it looks, because withholding data is not an attributable fault. If a producer publishes only part of the data and a node complains, other nodes cannot tell whether the producer withheld it or the node raised a false alarm, since the producer can release the data just after the complaint. Unlike an invalid block or a double vote, withholding cannot be proven to others, and so cannot be punished by slashing [Al-+21]. It must be detected instead, and every node must be able to detect it for itself. The same issue limits light clients. Fraud proofs protect them only if the data needed to build those proofs is available [Al-+21].

From calldata to blobs. Rollups first posted their data as *calldata*, the input field of ordinary layer-1 transactions. This data competed with every other transaction for block space, paid the same gas price, and stayed in the history forever, although rollups only need it for a few weeks. EIP-4844, also called Proto-Danksharding, deployed in March 2024, introduced a dedicated space [But+22]. Rollup data now travels in *blobs* of 128 KB, attached to blocks but not part of them. Blobs are committed to with KZG polynomial commitments [KZG10], cannot be read by the EVM, have their own fee market, and are deleted after 4,096 epochs, about 18 days. Every node still downloads every blob, however, which is why their number is kept small. The cap was 3 blobs per block as a target and 6 as a maximum at launch (0.375 and 0.75 MB), raised to 6 and 9 with the Pectra upgrade [Jay+24]. TODO *check the current values before submission (Fusaka's BPO forks were scheduled to raise them to 10/15, then 14/21).* As Section 2.1 showed, the bandwidth of ordinary nodes rules out

going much further this way.

Danksharding and data availability sampling. Danksharding is Ethereum's target design for data availability: up to 32 MB of blob data per block [Eth24]. At that volume, no node downloads everything, and each one stores only a part of the data. A node that does not see all the data must still decide whether it is available before attesting to the block. *Data availability sampling* (DAS) answers this with erasure coding [Al-+21]. The data is extended with redundancy so that any large enough fraction of it suffices to reconstruct all of it. An adversary who wants to make the data unrecoverable must then withhold a large fraction, and a node that requests a few randomly chosen pieces notices with high probability.

In Danksharding, the blob data is arranged in a 256×256 matrix of 512 B cells and extended with a two-dimensional Reed–Solomon code to a 512×512 matrix, in which any row or column can be rebuilt from half of its cells. To

make such data unrecoverable, an adversary must withhold at least 257^2 of the 512^2 cells, about 25% of them [Al-+21]. A node that samples s cells uniformly at random then misses the withholding with probability at most 0.75^s. The 73 samples planned for each node bring this below 10^{-9}. **Matthieu** *CHECK: own computation ($(1 - 257^2/512^2)^{73} \approx 6 \times 10^{-10}$, sampling with replacement); confirm this matches how the 73-sample target was derived.* **TODO** *cite the source of the 73-sample figure (PANDAS takes it from an Ethereum community DAS query-analysis notebook)*

Each cell carries a 48 B KZG proof that binds it to the commitment of the blob, so the extended data is about 140 MB per block. These proofs also settle a second question: whether the producer encoded the data correctly. Al-Bassam et al. handle incorrect encoding with fraud proofs [Al-+21], whereas KZG commitments rule it out by construction, since a cell that does not match the committed polynomial fails verification [KZG10]. Encoding 32 MB and sending out 140 MB every 12 seconds is far beyond an ordinary node. Dankshard-

ing therefore assigns this work to the builder, which proposer-builder separation already makes a powerful, well-connected actor (Section 1.3.5) [Hei+23; Fei22; Eth24]. Part IV relies on the builder for exactly this step.

The timing constraint. In Ethereum, availability must be decided within consensus. Attesters vote on a block within the first 4 seconds of its 12-second slot (Section 1.3.2). Under a *tight* fork-choice rule, a node checks availability before attesting, so a block is never voted for if its data is missing. A *trailing* rule would let availability be checked later, but an attestation could then have to be reverted when the data turns out to be missing [DZS24]. Changing consensus to allow this opens the door to ex-ante reorganization attacks [DZ24]. To leave consensus unchanged, the data must therefore be disseminated and sampled within those 4 seconds, by every node, for every block.

Separate data availability layers. Some systems provide data availability as a separate blockchain. DAS was introduced by Al-Bassam et al. [Al-+21]

and taken up by LazyLedger [Al-19], which became Celestia [Cel]. In Celestia, validators act as resourceful nodes that store the complete data, and light clients sample from them, so the cost of the validators grows linearly with the data volume and the number of samplers. ACeD instead has a group of oracle nodes store the coded data, assuming that at least half of them are honest [She+21b]. Other services such as EigenDA and Avail follow similar ideas, each with its own trust assumptions and timing. **Matthieu** *TODO: cite EigenDA and Avail or drop them.* A rollup that publishes its data on such a layer no longer gets the guarantees of Ethereum on its data, which is why we focus on data availability within Ethereum itself.

Networking for DAS in Ethereum. Within Ethereum, the open question is how to disseminate the cells and sample them over the peer-to-peer network. The building blocks available are GossipSub [Vyz+20], which disseminates blocks and attestations, and the Kademlia DHT [MM02], which sup-

ports node discovery. PeerDAS [Rya23; Rya+24] is the first step toward DAS, deployed with the Fusaka upgrade in December 2025. **Matthieu** *CHECK: Fusaka activation date and PeerDAS parameters (128 column subnets) at submission time.* It uses only one-dimensional, row-wise erasure coding and samples by *column*, a vertical slice across all blobs. Each of 128 GossipSub subnets carries one column, and nodes are statically assigned to the subnets whose columns they keep and check. This works for a moderate number of blobs, but dissemination by gossip does not scale to 32 MB per block. Every additional topic adds overhead, and redundant copies of messages waste bandwidth.

Other proposals go further, but remain incomplete. SubnetDAS adds two-dimensional coding and assigns each validator a row and a column rather than random samples [DD23]. Attesting only on what one was assigned weakens the guarantee, since a validator may attest although much of the data is missing. FullDAS combines gossip dissemination with request-response sampling

for large blobs, but its sampling process is only sketched. How custody is assigned, which transport is used, and how timing is met are left unspecified [KBR24]. Retrieving cells through the DHT is no better. Lookups take several hops, and seeding cells to nodes through Kademlia incurs high overhead [Cor+24]. **Matthieu** *CHECK: Cortes-Goicoechea et al. show DHT seeding overhead; "too slow for 4 s" may be stronger than what they claim. Also find a venue/URL for the bib entry.* DHTs are also exposed to Sybil and eclipse attacks [MHG18; Hen+19]. Honeybee makes peer sampling resistant to Sybils with verifiable random walks [ZV24], but checking each hop interactively adds latency that tight deadlines cannot afford. **Matthieu** *CHECK: Honeybee's arXiv title changed (v4: "Byzantine Tolerant Decentralized Peer Sampling with Verifiable Random Walks"); update bib if the final version is cited.*

What is missing. No existing design disseminates and samples data at the scale of Danksharding within the 4 seconds that Ethereum's consensus al-

lows. Such a design must also tolerate faulty and unresponsive nodes, and nodes that see different views of the network. It must stay within the hardware and bandwidth recommended for Ethereum nodes [Jay+25], and must not change the consensus protocol. Part IV presents PANDAS, which addresses this gap [Pig+25].

Synthesis and Motivation 3

Chapter 2 reviewed the main ways to scale blockchains and ended each review on what is still missing. This short chapter closes Part I by gathering those gaps and turning them into the research questions of this thesis. **Matthieu**

CHECK: RQ1–RQ3 are stated in Section 3.1; keep their wording identical to the Introduction once it is written.

Section 3.1 groups the limitations into three open challenges, one for each of the following parts. The first is to check, on real data, the workload assumptions that many designs rely on. The second is to place state across shards in a way that cuts cross-shard transactions while staying small, verifiable and resistant to manipulation. The third is to verify the availability of layer-2 data by sampling, within the timing of Ethereum's consensus. Sec-

tion 3.2 sets out what the solutions to the last two must share, whether they shard computation on layer 1 or data for layer 2. They must be verifiable by any node without a trusted party, yet flexible enough for large and changing systems, and they must stay decentralized and practical to deploy. Finally, Section 3.3 places the contributions relative to each other and gives the roadmap for Parts II to IV. SOAPS and PANDAS address different bottlenecks and complement each other.

3.1 Open Challenges

The limitations identified in Chapter 2 reduce to three open challenges. Each motivates one research question, and one part of this thesis.

C1: Knowing the workload. Many of the techniques reviewed in Section 2.2 and Section 2.3 rely, explicitly or not, on properties of the transaction workload. Parallel execution and sharding assume that concurrent transac-

tions mostly access disjoint state. Light nodes, stateless clients and snapshot-based bootstrap assume that recently used state is reused. Placement optimizations go further and assume that past interactions between accounts predict future ones. These assumptions are rarely measured on a real, large and recent workload, **Matthieu** *CHECK: Part II currently uses pre-Merge data; either update it with recent data (plan TODO) or drop "recent" here.* and designs are often evaluated on synthetic workloads or limited traces. **Matthieu** *CHECK: soften or support "rarely measured" and "often evaluated on synthetic workloads or limited traces" with a few examples from the related work.* Whether they hold, for which accounts, and whether they remain stable over time, determines how much these designs can actually gain. This leads to our first research question.

RQ1. Can we derive solid assumptions on the near-future workload of a blockchain from its historical transaction data?

C2: Placing state safely. State sharding splits execution and state across shards, but hash-based placement makes nearly every Ethereum transaction cross-shard (Section 2.3.3). Placement optimizations reduce this share, but assign every account explicitly, which is too much metadata for every node to keep at Ethereum's scale. Most of them also leave open who computes the placement and how others can check it (Section 2.3.4). In an open blockchain, a placement that a single party decides without verification invites selfish optimization, and can even help an attacker gather target accounts in a shard it controls. The designs that do make placement verifiable have every node or a whole committee redo the optimization. What is needed is a placement that reduces cross-shard transactions with bounded metadata, that any node can verify, and that fits into the existing consensus and its incentives without a trusted coordinator.

RQ2. How can state be placed in an account-based sharded blockchain

to reduce cross-shard transactions, without centralizing placement decisions or exposing them to selfish optimization?

C3: Checking data availability in time. Rollups move the pressure onto layer 1, which must make their data available (Section 2.4). Broadcasting all of it to every node does not scale beyond a few blobs per block, and data availability sampling lets each node check availability while downloading only a small part. In Ethereum, however, sampling must complete within the first 4 seconds of a slot, so that attesters can vote on the block and on the availability of its data at the same time. It must work over an open peer-to-peer network of thousands to tens of thousands of nodes, some of them faulty, unresponsive or with an incomplete view of the network, and without adding a step to consensus. Existing proposals either do not scale to the target data volume, weaken the guarantees of sampling, or leave the sampling process unspecified (Section 2.4.4).

RQ3. How can layer-2 data be disseminated and sampled within Ethereum's consensus time bounds, in a fully decentralized, peer-to-peer way?

3.2 Common Requirements for L1 and L2 Sharding

Challenges C2 and C3 concern different resources: execution and state on layer 1, and the data that layer-2 protocols publish on it. Both are forms of sharding, however, since both make each node responsible for only part of the system. They share a central difficulty: deciding *who is responsible for what*, accounts for shards or data for nodes, in an open system where some participants are adversarial. We derive four requirements that our solutions to both challenges must meet.

Verifiable. Any node must be able to recompute or check, on its own, who is assigned what, without trusting a third party. Ethereum already solves this

problem for its committees. The assignment of validators is a deterministic function of public randomness and public state, so any node can recompute it and reject a vote from the wrong committee (Section 1.3.3). Our designs reuse this principle. SOAPS elects, from the beacon chain's randomness, the node that computes the placement and those that check it. PANDAS derives from the epoch seed the part of the data that each node keeps.

Flexible. Determinism alone is not enough. A fully static assignment, such as hash-based placement, cannot follow the workload, and a fixed communication plan cannot cope with nodes that fail or that see the network differently. A fully ad hoc approach, on the other hand, cannot be checked. Our designs sit in between. They fix the part of the decision that must be verifiable, and leave a bounded part free to adapt. In SOAPS, a limited set of explicit placement rules is recomputed periodically from the workload. In PANDAS, nodes adapt how they fetch data to the conditions they observe.

Decentralized. The solutions must not concentrate power or cost. They must add no heavy consensus step, such as running a BFT agreement on every placement update, and keep the cost of an ordinary node within the hardware and bandwidth recommended for Ethereum [Jay+25]. Anyone must remain able to participate.

Practical. The solutions must be grounded in how Ethereum actually works. They should be evaluated on real Ethereum workloads and under its consensus timing, and remain compatible with the existing protocols, rather than assume a blockchain designed from scratch.

3.3 Positioning of the Contributions

Figure 3.1 places the contributions of this thesis along two axes: the layer concerned, and whether what is sharded is computation or data.

SOAPS (Part III) shards computation on layer 1. It makes state shard-

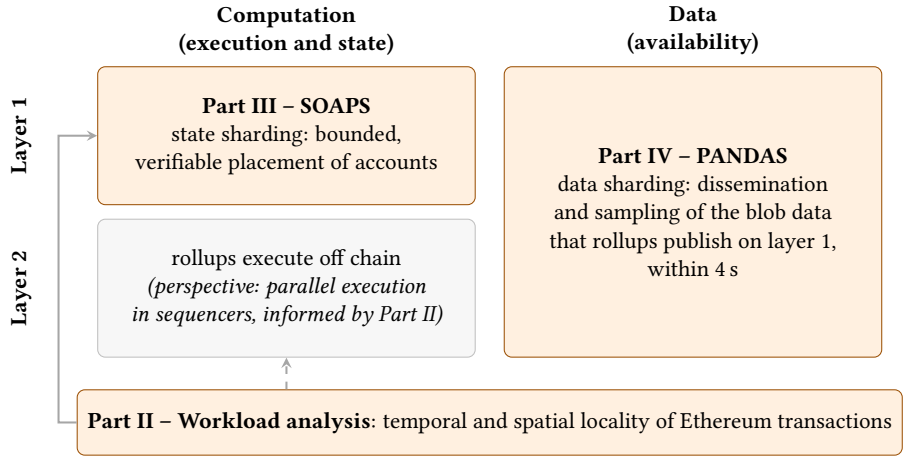


Figure 3.1: Positioning of the contributions. SOAPS (Part III) shards computation on layer 1 and builds on the workload analysis of Part II. PANDAS (Part IV) shards the data that layer-2 protocols publish on layer 1. Parallel execution in rollup sequencers is left as a perspective.

ing practical by placing explicitly only a bounded set of the accounts that cause the most cross-shard transactions, and by integrating the computation and verification of this placement into consensus. Its benefit depends on the workload, and it builds directly on Part II, which identifies the small set of accounts responsible for most of the loss of spatial locality.

PANDAS (Part IV) shards data. It removes the bandwidth bottleneck of data availability, so that layer 1 can host far more layer-2 data, while each node handles only a small share of it. It depends very little on the workload,

since erasure-coded cells carry no semantic locality. Its difficulty lies instead in the network: latency, failures, and nodes with inconsistent views, under a hard deadline.

The two contributions complement each other rather than compete. A sharded layer 1 still has to host the data of the rollups built on it, and the data of each shard could itself be sampled. Conversely, scaling data availability does not make execution on layer 1 cheaper. The workload analysis of Part II informs the first, and, beyond this thesis, could inform parallel execution in rollup sequencers.

The rest of this thesis follows this structure. Part II analyzes the locality of Ethereum transactions and answers RQ1 [PKR23]. Part III presents SOAPS and answers RQ2. **Matthieu** *TODO: cite the SOAPS paper here (and in the Part III row of Table 2.1) once it has a bib entry.* Part IV presents PANDAS and answers RQ3 [Pig+25]. The conclusion compares the two designs, summarizes the

answers to the research questions, and discusses perspectives.

Bibliography

[AAE19] M. J. Amiri, D. Agrawal, and A. El Abbadi. "ParBlockchain: Leveraging Transaction Parallelism in Permissioned Blockchain Systems". In: *IEEE ICDCS*. 2019.

[Adh+25] R. Adhikari, C. Busch, D. R. Kowalski, and A. Al-Mamun. "Transaction Processing in Blockchain Sharding: Current Trends and Future Research Directions". In: *Distributed Ledger Technologies: Research and Practice* (2025).

[Al +24] M. M. Al Barat, S. Li, C. Du, Y. T. Hou, and W. Lou. "SoK: Public Blockchain Sharding". In: *IEEE ICBC*. 2024, pp. 766–783.

[Al-+18] M. Al-Bassam, A. Sonnino, S. Bano, D. Hrycyszyn, and G. Danezis. "Chainspace: A Sharded Smart Contracts Platform". In: *NDSS*. 2018.

[Al-+21] M. Al-Bassam, A. Sonnino, V. Buterin, and I. Khoffi. "Fraud and Data Availability Proofs: Detecting Invalid Blocks in Light Clients". In: *Financial Cryptography and Data Security (FC)*. 2021, pp. 279–298.

[Al-19] M. Al-Bassam. "LazyLedger: A Distributed Data Availability Ledger with Client-Side Smart Contracts". In: *arXiv preprint arXiv:1905.09274* (2019).

[And+18] E. Androulaki, A. Barger, V. Bortnikov, C. Cachin, K. Christidis, A. De Caro, D. Enyeart, C. Ferris, G. Laventman, Y. Manevich, et al. "Hyperledger Fabric: A Distributed Operating System for Permissioned Blockchains". In: *ACM EuroSys*. 2018.

[Anj+22] P. S. Anjana, S. Kumari, S. Peri, S. Rathor, and A. Somani. "OptSmart: A Space Efficient Optimistic Concurrent Execution of Smart Contracts". In: *Distributed and Parallel Databases* (2022).

[Ava+23] Z. Avarikioti, A. Desjardins, L. Kokoris-Kogias, and R. Wattenhofer. "Divide & scale: Formalization and roadmap to robust sharding". In: *International Colloquium on Structural Information and Communication Complexity*. Springer. 2023, pp. 199–245.

[Bla+24] S. Blackshear et al. "Sui Lutris: A Blockchain Combining Broadcast and Consensus". In: *ACM CCS*. 2024, pp. 2606–2620.

[Blo+08] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre. "Fast Unfolding of Communities in Large Networks". In: *Journal of Statistical Mechanics: Theory and Experiment* 2008.10 (2008), P10008.

[BLS01] D. Boneh, B. Lynn, and H. Shacham. "Short Signatures from the Weil Pairing". In: *Advances in Cryptology (ASIACRYPT)*. 2001, pp. 514–532.

[Bon+03] D. Boneh, C. Gentry, B. Lynn, and H. Shacham. "Aggregate and Verifiably Encrypted Signatures from Bilinear Maps". In: *Advances in Cryptology (EUROCRYPT)*. Vol. 2656. Lecture Notes in Computer Science. Springer, 2003, pp. 416–432. DOI: 10.1007/3-540-39200-9_26.

[BS20] V. Buterin and M. Swende. *EIP-2930: Optional Access Lists*. 2020. URL: https://eips.ethereum.org/EIPS/eip-2930 (visited on 10/16/2026).

[But+19] V. Buterin, E. Conner, R. Dudley, M. Slipper, I. Norden, and A. Bakhta. *EIP-1559: Fee Market Change for ETH 1.0 Chain*. 2019. URL:

https://eips.ethereum.org/EIPS/eip-1559 (visited on 10/16/2026).

[But+22] V. Buterin, D. Feist, D. Loerakker, G. Kadianakis, M. Garnett, M. Taiwo, and A. Dietrichs. *EIP-4844: Shard Blob Transactions*. 2022. URL: https://eips.ethereum.org/EIPS/eip-4844 (visited on 10/16/2026).

[But20] V. Buterin. *A Rollup-Centric Ethereum Roadmap*. Ethereum Magicians. 2020. URL: https://ethereum-magicians.org/t/a-rollup-centric-ethereum-roadmap/4698 (visited on 10/16/2026).

[CBC22] P. Chatzigiannis, F. Baldimtsi, and K. Chalkias. "SoK: Blockchain Light Clients". In: *Financial Cryptography and Data Security (FC)*. 2022, pp. 615–641.

[Cel] Celestia. *Celestia: The First Modular Blockchain Network*. URL: https://celestia.org/ (visited on 10/16/2026).

[Cha+20] M. M. T. Chakravarty, J. Chapman, K. MacKenzie, O. Melkonian, M. Peyton Jones, and P. Wadler. "The Extended UTXO Model". In: *Financial Cryptography and Data Security (FC 2020 Workshops)*. Vol. 12063. Lecture Notes in Computer Science. Springer, 2020, pp. 525–539. DOI: 10.1007/978-3-030-54455-3_37.

[Che+21] Y. Chen, Z. Guo, R. Li, S. Chen, L. Zhou, Y. Zhou, and X. Zhang. "Forerunner: Constraint-Based Speculative Transaction Execution for Ethereum". In: *ACM SOSP*. 2021.

[CKR20] S. Cao, S. Kadhe, and K. Ramchandran. "CoVer: Collaborative Light-Node-Only Verification and Data Availability for Blockchains". In: *IEEE Blockchain*. 2020.

[CL99] M. Castro and B. Liskov. “Practical Byzantine Fault Tolerance”. In: *USENIX OSDI*. 1999, pp. 173–186.

[Cor+24] M. Cortes-Goicoechea, C. Kiraly, D. Ryajov, J. L. Muñoz-Tapia, and L. Bautista-Gomez. *Scalability Limitations of Kademlia DHTs when Enabling Data Availability Sampling in Ethereum*. 2024.

[Cro+16] K. Croman, C. Decker, I. Eyal, A. E. Gencer, A. Juels, A. Kosba, A. Miller, P. Saxena, E. Shi, E. Gün Sirer, D. Song, and R. Wattenhofer. “On Scaling Decentralized Blockchains”. In: *Financial Cryptography and Data Security (FC) Workshops*. Lecture Notes in Computer Science. Springer, 2016, pp. 106–125.

[Cur+10] C. Curino, E. P. C. Jones, Y. Zhang, and S. R. Madden. “Schism: a workload-driven approach to database replication and partitioning”. In: *Proc. VLDB Endow.* (2010).

[Dai+20] P. Daian, S. Goldfeder, T. Kell, Y. Li, X. Zhao, I. Bentov, L. Breidenbach, and A. Juels. "Flash Boys 2.0: Frontrunning in Decentralized Exchanges, Miner Extractable Value, and Consensus Instability". In: *IEEE S&P*. 2020, pp. 910–927.

[DD23] F. D'Amato and A. Dietrichs. *SubnetDAS – An Intermediate DAS Approach*. Ethereum Research. 2023. URL: https://ethresear.ch/t/subnetdas-an-intermediate-das-approach/17169 (visited on 10/16/2026).

[Dem+87] A. Demers, D. Greene, C. Hauser, W. Irish, J. Larson, S. Shenker, H. Sturgis, D. Swinehart, and D. Terry. "Epidemic Algorithms for Replicated Database Maintenance". In: *ACM PODC*. 1987, pp. 1–12.

[Den68] P. J. Denning. "The Working Set Model for Program Behavior". In: *Communications of the ACM* 11.5 (1968), pp. 323–333.

[Dic+17] T. Dickerson, P. Gazzillo, M. Herlihy, and E. Koskinen. "Adding Concurrency to Smart Contracts". In: *ACM PODC*. 2017, pp. 303–312.

[DLS88] C. Dwork, N. Lynch, and L. Stockmeyer. "Consensus in the Presence of Partial Synchrony". In: *Journal of the ACM* 35.2 (1988), pp. 288–323.

[DN92] C. Dwork and M. Naor. "Pricing via Processing or Combatting Junk Mail". In: *Advances in Cryptology (CRYPTO)*. 1992, pp. 139–147.

[Dou02] J. R. Douceur. "The Sybil Attack". In: *Peer-to-Peer Systems (IPTPS)*. 2002, pp. 251–260.

[DZ24] F. D'Amato and L. Zanolini. "Recent Latest Message Driven GHOST: Balancing Dynamic Availability with Asynchrony Resilience". In: *IEEE CSF*. 2024, pp. 127–142.

[DZS24] F. D'Amato, L. Zanolini, and R. Saltini. *DAS Fork-Choice*. Ethereum Research. May 2024. URL: https://ethresear.ch/t/das-fork-choice/19578 (visited on 10/16/2026).

[Etha] Ethereum Foundation. *Ethereum Proof-of-Stake Consensus Specifications*. URL: https://github.com/ethereum/consensus-specs (visited on 10/12/2026).

[Ethb] Ethereum Foundation. *Optimistic Rollups*. URL: https://ethereum.org/en/developers/docs/scaling/optimistic-rollups/ (visited on 10/16/2026).

[Ethc] Ethereum Foundation. *Zero-Knowledge Rollups*. URL: https://ethereum.org/en/developers/docs/scaling/zk-rollups/ (visited on 10/16/2026).

[Ethd] Etherscan. *Ethereum Average Gas Limit Chart*. URL: https://etherscan.io/chart/gaslimit (visited on 10/16/2026).

[Ethe] Etherscan. *Ethereum Daily Transactions Chart*. URL: https://etherscan.io/chart/tx (visited on 10/16/2026).

[Eth24] Ethereum Foundation. *Danksharding*. 2024. URL: https://ethereum.org/en/roadmap/danksharding/ (visited on 10/16/2026).

[Fei22] D. Feist. *New Sharding Design with Tight Beacon and Shard Block Integration*. 2022. URL: https://notes.ethereum.org/@dankrad/new_sharding (visited on 10/16/2026).

[FLP85] M. J. Fischer, N. A. Lynch, and M. S. Paterson. "Impossibility of Distributed Consensus with One Faulty Process". In: *J. ACM* 32.2 (1985), pp. 374–382.

[FP18] E. Fynn and F. Pedone. "Challenges and Pitfalls of Partitioning Blockchains". In: *IEEE/IFIP DSN Workshops*. 2018, pp. 128–133.

[Fyn+22] E. Fynn, E. Buchman, Z. Milosevic, R. Soulé, and F. Pedone. "Robust and Fast Blockchain State Synchronization". In: *OPODIS*. 2022.

[GDW19] D. Guo, J. Dong, and K. Wang. "Graph structure and statistical properties of Ethereum transaction relationships". In: *Inf. Sci.* 492 (2019), pp. 58–71.

[Gel+23] R. Gelashvili, A. Spiegelman, Z. Xiang, G. Danezis, Z. Li, D. Malkhi, Y. Xia, and R. Zhou. "Block-STM: Scaling Blockchain Execution by Turning Ordering Curse to a Performance Blessing". In: *ACM PPoPP*. 2023, pp. 232–244.

[Ger+16] A. Gervais, G. O. Karame, K. Wüst, V. Glykantzis, H. Ritzdorf, and S. Capkun. "On the Security and Performance of Proof of Work Blockchains". In: *ACM CCS*. 2016, pp. 3–16.

[Gil+17] Y. Gilad, R. Hemo, S. Micali, G. Vlachos, and N. Zeldovich. "Algorand: Scaling Byzantine Agreements for Cryptocurrencies". In: *ACM SOSP*. 2017, pp. 51–68.

[GKL15] J. Garay, A. Kiayias, and N. Leonardos. "The Bitcoin Backbone Protocol: Analysis and Applications". In: *EUROCRYPT*. 2015, pp. 281–310.

[Gud+20] L. Gudgeon, P. Moreno-Sanchez, S. Roos, P. McCorry, and A. Gervais. "SoK: Layer-Two Blockchain Protocols". In: *Financial Cryptography and Data Security (FC)*. 2020, pp. 201–226.

[Hai+25] M. Z. Haider, T. Noreen, M. D. Assuncao, and K. Zhang. "AI-Driven Predictive Shard Allocation for Scalable Next Generation Blockchains". In: *arXiv preprint arXiv:2511.19450* (2025).

[Hei+23] L. Heimbach, L. Kiffer, C. Ferreira Torres, and R. Wattenhofer. "Ethereum's Proposer-Builder Separation: Promises and Realities". In: *ACM IMC*. 2023, pp. 406–420.

[Hen+19] S. Henningsen, D. Teunis, M. Florian, and B. Scheuermann. "Eclipsing Ethereum Peers with False Friends". In: *IEEE EuroS&P Workshops*. 2019, pp. 300–309.

[HLZ24] H. Huang, Y. Lin, and Z. Zheng. "Account migration across blockchain shards using fine-tuned lock mechanism". In: *IEEE INFOCOM 2024*. 2024, pp. 271–280.

[HS91] S. Haber and W. S. Stornetta. "How to Time-Stamp a Digital Document". In: *Journal of Cryptology* 3.2 (1991), pp. 99–111.

[Hua+22] H. Huang, X. Peng, J. Zhan, S. Zhang, Y. Lin, Z. Zheng, and S. Guo. "Brokerchain: A cross-shard blockchain protocol for account/balance-based state sharding". In: *IEEE INFOCOM*. 2022, pp. 1968–1977.

[Hua+26] H. Huang, J. Zheng, X. Zhu, Y. Lin, T. Cai, L. Zhou, Z. Zheng, and S. Guo. "ShardCutter: A Blockchain Sharding Protocol Achieving Transaction Workload Balance across State Shards". In: *IEEE Transactions on Networking* (2026).

[Jay+24] P. Jayanthi, T. Wahrstätter, S. Calder-Mason, A. Davis, and A. Dietrichs. *EIP-7691: Blob Throughput Increase*. 2024. URL: https://eips.ethereum.org/EIPS/eip-7691 (visited on 10/16/2026).

[Jay+25] P. Jayanthi, K. Wedderburn, J. Rudolf, D. Feist, J. Traglia, I. Hagopian, G. Kadianakis, F. Svantes, C. Beekhuizen, and T. Wahrstätter. *EIP-7870: Hardware and Bandwidth Recommendations*. 2025. URL: https://eips.ethereum.org/EIPS/eip-7870 (visited on 10/16/2026).

[Kal+18] H. Kalodner, S. Goldfeder, X. Chen, S. M. Weinberg, and E. W. Felten. "Arbitrum: Scalable, private smart contracts". In: *USENIX Security*. 2018, pp. 1353–1370.

[KBR24] C. Kiraly, L. Bautista-Gomez, and D. Ryajov. *FullDAS: Towards Massive Scalability with 32MB Blocks and Beyond*. Ethereum Research. 2024. URL: https://ethresear.ch/t/fulldas-towards-massive-scalability-with-32mb-blocks-and-beyond/19529 (visited on 10/16/2026).

[Kia+17] A. Kiayias, A. Russell, B. David, and R. Oliynykov. "Ouroboros: A Provably Secure Proof-of-Stake Blockchain Protocol". In: *CRYPTO*. 2017, pp. 357–388.

[Kim+21] J.-Y. Kim, J. Lee, Y. Koo, S. Park, and S.-M. Moon. "Ethanos: Efficient Bootstrapping for Full Nodes on Account-Based Blockchain". In: *ACM EuroSys*. 2021.

[KK98] G. Karypis and V. Kumar. "A fast and high quality multilevel scheme for partitioning irregular graphs". In: *SIAM J. Sci. Comput.* 20.1 (1998), pp. 359–392.

[Kok+18] E. Kokoris-Kogias, P. Jovanovic, L. Gasser, N. Gailly, E. Syta, and B. Ford. "OmniLedger: A Secure, Scale-Out, Decentralized Ledger via Sharding". In: *IEEE S&P*. 2018, pp. 583–598.

[Kró+21] M. Król, O. Ascigil, S. Rene, A. Sonnino, M. Al-Bassam, and E. Rivière. "Shard scheduler: Object placement and migration in sharded account-based blockchains". In: *ACM AFT*. 2021, pp. 43–56.

[KZG10] A. Kate, G. M. Zaverucha, and I. Goldberg. "Constant-Size Commitments to Polynomials and Their Applications". In: *ASIACRYPT*. 2010, pp. 177–194.

[Li+22] C. Li, H. Huang, Y. Zhao, X. Peng, R. Yang, Z. Zheng, and S. Guo. “Achieving Scalability and Load Balance across Blockchain Shards for State Sharding”. In: *IEEE SRDS*. 2022, pp. 284–294.

[Li+24] P. Li, M. Song, M. Xing, Z. Xiao, Q. Ding, S. Guan, and J. Long. “Spring: Improving the throughput of sharding blockchain via deep reinforcement learning based state placement”. In: *ACM Web Conf.* 2024, pp. 2836–2846.

[Liu+26] Y. Liu, L. Jia, X. Yang, Z. Li, and Y. Sun. “Concordia: Enabling Low-Conflict Distributed Transaction Scheduling in Sharding Blockchain via Cooperative Perception”. In: *ACM Web Conf.* 2026, pp. 5493–5502.

[LNS25] J. Li, Y. Ning, and X. Song. “A Novel Blockchain Transaction Sharding Scheme for Low Cross-Shard and Adaptive Load Bal-

ance". In: *IEEE Transactions on Network and Service Management* (2025).

[LSP82] L. Lamport, R. Shostak, and M. Pease. "The Byzantine Generals Problem". In: *ACM Trans. Program. Lang. Syst.* 4.3 (1982), pp. 382–401.

[Luu+16] L. Luu, V. Narayanan, C. Zheng, K. Baweja, S. Gilbert, and P. Saxena. "A Secure Sharding Protocol For Open Blockchains". In: *ACM CCS*. 2016, pp. 17–30.

[Meh+25] B. Mehta, N. Baghel, M. J. Amiri, B. T. Loo, and R. Marcus. "Adaptive Sharding in Untrusted Environments". In: *Proc. ACM Manag. Data* 3.6 (2025), pp. 1–28.

[Mer87] R. C. Merkle. "A Digital Signature Based on a Conventional Encryption Function". In: *Advances in Cryptology (CRYPTO)*. 1987, pp. 369–378.

[MHG18] Y. Marcus, E. Heilman, and S. Goldberg. *Low-Resource Eclipse Attacks on Ethereum's Peer-to-Peer Network*. IACR Cryptology ePrint Archive, Paper 2018/236. 2018.

[MM02] P. Maymounkov and D. Mazières. "Kademlia: A Peer-to-Peer Information System Based on the XOR Metric". In: *Peer-to-Peer Systems (IPTPS)*. 2002, pp. 53–65.

[MR20] A. Mizrahi and O. Rottenstreich. "Blockchain State Sharding with Space-Aware Representations". In: *IEEE Transactions on Network and Service Management* 18.2 (2020), pp. 1571–1583.

[MW23] K. Mu and X. Wei. "EfShard: Toward Efficient State Sharding Blockchain via Flexible and Timely State Allocation". In: *IEEE Transactions on Network and Service Management* 20.3 (2023), pp. 2817–2829.

[Nag+25] Á. Nagy, J. Tapolcai, I. A. Seres, and B. Ladóczki. "Forking the RANDAO: Manipulating Ethereum's Distributed Randomness Beacon". In: *ACM CCS*. 2025, pp. 873–887.

[Nak08] S. Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System*. `https://bitcoin.org/bitcoin.pdf`. 2008.

[Ngu+19] L. N. Nguyen, T. D. T. Nguyen, T. N. Dinh, and M. T. Thai. "OptChain: Optimal Transactions Placement for Scalable Blockchain Sharding". In: *IEEE ICDCS*. 2019, pp. 525–535.

[NTT21] J. Neu, E. N. Tas, and D. Tse. "Ebb-and-Flow Protocols: A Resolution of the Availability-Finality Dilemma". In: *IEEE S&P*. 2021, pp. 446–465.

[Opt] Optimism. *OP Stack Specification*. URL: `https://specs.optimism.io/` (visited on 10/16/2026).

[Pac+23] M. Pacheco, G. Oliva, G. K. Rajbahadur, and A. Hassan. "Is my transaction done yet? an empirical study of transaction processing times in the ethereum blockchain platform". In: *ACM Trans. Softw. Eng. Methodol.* 32.3 (2023), pp. 1–46.

[PAT24] U. Pavloff, Y. Amoussou-Guenou, and S. Tucci-Piergiovanni. "Byzantine attacks exploiting penalties in Ethereum PoS". In: *IEEE/IFIP DSN*. 2024, pp. 53–65.

[PCG21] S. Paavolainen, C. Carr, and E. Ghadafi. "Adventures of a Light Blockchain Protocol in a Forest of Transactions: A Subset of a Story". In: *IEEE Access* (2021).

[PD16] J. Poon and T. Dryja. *The Bitcoin Lightning Network: Scalable Off-Chain Instant Payments*. Technical report. 2016.

[Pig+25] M. Pigaglio, O. Ascigil, M. Król, F. Lange, K. Peeroo, S. Rene, R. Sadre, V. Stankovic, and E. Rivière. "PANDAS: Peer-to-peer,

Adaptive Networking Allowing Data Availability Sampling within Ethereum Consensus Timebounds". In: *Middleware*. 2025, pp. 167–179.

[PKR23] M. Pigaglio, M. Król, and E. Rivière. "Exploring locality in Ethereum transactions". In: *IEEE BRAINS*. 2023, pp. 1–8.

[Qi+26] J. Qi, D. Ding, J. Li, J. Cao, Y.-C. Chen, G. Xue, and S. Liu. "BIND: Enabling Continuous Transaction Processing During Account Migration in Sharded Blockchains". In: *ACM Web Conf.* 2026, pp. 5547–5556.

[Rou24] T. Roughgarden. "Transaction Fee Mechanism Design". In: *J. ACM* (2024), pp. 1–25.

[Rya+24] D. Ryan, D. Feist, F. D'Amato, H.-W. Wang, and A. Stokes. *EIP-7594: PeerDAS – Peer Data Availability Sampling*. 2024. URL: `https://eips.ethereum.org/EIPS/eip-7594` (visited on 10/16/2026).

[Rya23] D. Ryan. *PeerDAS – A Simpler DAS Approach Using Battle-Tested P2P Components*. Ethereum Research. 2023. URL: https://ethresear.ch/t/peerdas-a-simpler-das-approach-using-battle-tested-p2p-components/16541 (visited on 10/16/2026).

[Sch+22] C. Schwarz-Schilling, J. Neu, B. Monnot, A. Asgaonkar, E. N. Tas, and D. Tse. "Three Attacks on Proof-of-Stake Ethereum". In: *Financial Cryptography and Data Security (FC)*. 2022, pp. 560–576.

[Sch90] F. B. Schneider. "Implementing Fault-Tolerant Services Using the State Machine Approach: A Tutorial". In: *ACM Computing Surveys* 22.4 (1990), pp. 299–319.

[She+21a] P. Sheng, G. Wang, K. Nayak, S. Kannan, and P. Viswanath. "BFT Protocol Forensics". In: *ACM CCS*. 2021, pp. 1722–1743.

[She+21b] P. Sheng, B. Xue, S. Kannan, and P. Viswanath. "ACeD: Scalable Data Availability Oracle". In: *Financial Cryptography and Data Security (FC)*. 2021.

[SZ15] Y. Sompolinsky and A. Zohar. "Secure High-Rate Transaction Processing in Bitcoin". In: *Financial Cryptography and Data Security (FC)*. 2015, pp. 507–527.

[TDL22] M. Toulouse, H. K. Dai, and T. G. Le. "Distributed Load-Balancing for Account-Based Sharded Blockchains". In: *International Journal of Web Information Systems* 18.2/3 (2022), pp. 100–116.

[Vyz+20] D. Vyzovitis, Y. Napora, D. McCormick, D. Dias, and Y. Psaras. "GossipSub: Attack-Resilient Message Propagation in the Filecoin and ETH2.0 Networks". In: *arXiv preprint arXiv:2007.02754* (2020).

[Wan+19] G. Wang, Z. J. Shi, M. Nixon, and S. Han. "SoK: Sharding on Blockchain". In: *ACM AFT*. 2019, pp. 41–61.

[WG24] Q. Wang and Y. Guan. "TransShard: A Dynamic Transaction-Aware Sharding Scheme for Account-Based Blockchain". In: *IEEE Access* (2024).

[Woo14] G. Wood. *Ethereum: A Secure Decentralised Generalised Transaction Ledger*. Ethereum Yellow Paper, `https://ethereum.github.io/yellowpaper/paper.pdf`. 2014.

[WW19] J. Wang and H. Wang. "Monoxide: Scale out blockchains with asynchronous consensus zones". In: *USENIX NSDI*. 2019, pp. 95–112.

[Yak19] A. Yakovenko. *Sealevel: Parallel Processing Thousands of Smart Contracts*. Solana Labs. 2019. URL: `https://medium.com/`

`solana-labs/sealevel-parallel-processing-thousands-of-smart-contracts-d814b378192` (visited on 10/16/2026).

[Yin+19] M. Yin, D. Malkhi, M. K. Reiter, G. G. Gueta, and I. Abraham. "HotStuff: BFT Consensus with Linearity and Responsiveness". In: *ACM PODC*. 2019, pp. 347–356.

[Zha+22] Z. Zhang, X. Wang, G. Yu, W. Ni, R. P. Liu, N. Georgalas, and A. Reeves. "A Community Detection-Based Blockchain Sharding Scheme". In: *ICBC*. 2022, pp. 78–91.

[Zha+25] Z. Zhang, H. Han, Z. Yan, and E. Gelenbe. "A Survey on Cross-Shard Transaction Allocation Optimization and Processing Mechanisms". In: *IEEE Communications Surveys & Tutorials* (2025).

[ZMR18] M. Zamani, M. Movahedi, and M. Raykova. "RapidChain: Scaling Blockchain via Full Sharding". In: *ACM CCS*. 2018, pp. 931–948.

[ZPY23] Y. Zhang, S. Pan, and J. Yu. "TxAllo: Dynamic Transaction Allocation in Sharded Blockchain Systems". In: *IEEE ICDE*. 2023, pp. 721–733.

[ZV24] Y. Zhang and S. B. Venkatakrishnan. "Honeybee: Decentralized Peer Sampling with Verifiable Random Walks for Blockchain Data Sharding". In: *arXiv preprint arXiv:2402.16201* (2024).